

GOETHE-UNIVERSITÄT FRANKFURT AM MAIN

FACHBEREICH INFORMATIK

Instruction Robustness in RAG/GraphRAG Agent Pipelines

Bachelor Thesis

Name: Christian Block

Matriculation Number: 6522508

First examiner: Prof. Dr. Visvanathan Ramesh

Date of Submission:

Erklärung zur Abschlussarbeit

gemäß § 35, Abs. 16 der Ordnung für den Bachelorstudiengang Informatik vom 17. Juni 2019:

Hiermit erkläre ich (Nachname, Vorname) **Block, Christian**, dass ich die vorliegende Arbeit selbstständig und ohne Benutzung anderer als der angegebenen Quellen und Hilfsmittel verfasst habe. Ich bestätige außerdem, dass die vorliegende Arbeit nicht, auch nicht auszugsweise, für eine andere Prüfung oder Studienleistung verwendet wurde. Zudem versichere ich, dass alle eingereichten schriftlichen gebundenen Versionen meiner vorliegenden Bachelorarbeit mit der digital eingereichten elektronischen Version meiner Bachelorarbeit übereinstimmen.

Frankfurt am Main, den DATUM

Unterschrift der/des Studierenden

Abstract

Retrieval-augmented generation (RAG) combines information retrieval with language generation, so that a language model can condition its output on external and updatable evidence rather than only on what it learned during training. The system prompt specifies the instructions, constraints, and output format the generator should follow. Because it is written in natural language, its effect on system behaviour is hard to predict and hard to verify. Prior work on prompt sensitivity in language models has studied that effect almost entirely outside retrieval-augmented systems. This thesis addresses that gap by asking how far systematic changes in the system prompt influence the behaviour and outcomes of a RAG system when the user input, knowledge base, retrieval mechanism, and underlying language model are held constant.

The thesis builds a formal model of a RAG system on the formulation of Lewis et al. (2020). A retriever assigns probabilities $p_\eta(z \mid q, D)$ to documents from the knowledge base, a generator produces the answer token by token. The system prompt S enters as an explicit conditioning variable, $p_\theta(y_i \mid q, z, y_{1:i-1}, S)$, and is modelled as a set of checkable requirements Σ written in a wording φ , such that $S = \rho(\Sigma, \varphi)$. Each of three published baseline prompts is written in four wordings: its canonical form and three controlled rewordings, which change the vocabulary, the sentence structure or the layout. Every rewording leaves the requirements fixed and changes only φ .

Changing the system prompt is an intervention on the system. What counts as a good answer is fixed beforehand. An answer must be grounded in the retrieved evidence, relevant to the question asked, correct against the reference answer, and obedient to the instructions it was given. The knowledge base and the query set come from established question-answer datasets, which keeps the measurement reproducible and its results comparable with published work. Only the system prompt is manipulated. The language model, the embedding model, the vector index, the chunking strategy and the retrieval depth stay fixed, so the passages that reach the generator are identical in every condition and any difference in the output originates at the generator alone. Generations are repeated to account for decoding randomness.

The twelve prompts were run over 100 MS MARCO queries at two evidence levels with three repetitions, which produced 6,768 answers. Rewording changed what the system did. On instruction adherence, eleven of twelve estimates exclude zero after correction for multiple comparisons, ten of them at both evidence levels, and the largest is 46 points. The direction is not fixed. Every rewording of one baseline scored above its published form and every rewording of another scored below it, so wording is a parameter that can be set well or badly rather than a hazard to be guarded against. No effect survives correction on faithfulness or answer relevance, but both sat near the top of their range in every condition, so this design cannot say whether they are affected. Correctness produced a result about measurement rather than about prompts: the measure fixed before the run detects nothing, while a second measure of the same property on the same answers detects thirteen effects out of eighteen. Both are reported. The hypothesis that prompts stating the same requirements in different words produce the same behaviour is rejected.

Zusammenfassung

Retrieval-Augmented-Generation-Systeme (RAG-Systeme) verbinden Informationsretrieval mit Sprachgenerierung. Ein Sprachmodell kann seine Ausgabe damit auf externe Evidenz stützen und muss sich nicht ausschließlich auf während des Trainings erlernte Parameter verlassen. Der System-Prompt legt innerhalb eines solchen Systems die Ausgabeanforderungen fest, denen der Generator folgen soll. Da er in natürlicher Sprache formuliert ist, lässt sich seine Wirkung auf das Systemverhalten nur schwer vorhersagen und überprüfen, und bisherige Arbeiten zur Prompt-Sensitivität von Sprachmodellen wurden fast ausschließlich außerhalb von Retrieval-Augmented-Systemen durchgeführt. Die vorliegende Arbeit adressiert diese Lücke und untersucht, in welchem Ausmaß systematische Änderungen des System-Prompts das Verhalten und die Ergebnisse eines RAG-Systems verändern, wenn Nutzereingabe, Wissensbasis, Retrieval-Mechanismus und zugrunde liegendes Sprachmodell konstant gehalten werden.

Die Arbeit entwickelt ein formales Modell eines RAG-Systems auf Grundlage der Formulierung von Lewis et al. (2020). Darin weist ein Retriever den dokumenten Wahrscheinlichkeiten $p_{\eta}(z \mid q, D)$ zu, und ein Generator erzeugt die Antwort Token für Token. Der System-Prompt wird als freie Variable $p_{\theta}(y_i \mid q, z, y_{1:i-1}, S)$ eingeführt und als eine Menge überprüfbarer Anforderungen Σ in einer Formulierung φ modelliert. Drei publizierte Baseline-Prompts werden in jeweils vier Formulierungen verwendet: der kanonischen Form und drei kontrollierten Umformulierungen, die Wortwahl, Satzbau oder Layout verändern, ohne die Anforderungen zu verändern. Die Änderung des System-Prompts wird als Intervention auf dieses Modell behandelt. Was als gute Antwort gilt, wird vorher festgelegt. Eine Antwort muss vier Anforderungen zugleich erfüllen, nämlich in der abgerufenen Evidenz verankert, für die gestellte Frage relevant, gegenüber der Referenzantwort korrekt und den erteilten Anweisungen gehorsam zu sein. Wissensbasis und Anfragemenge stammen aus etablierten Frage-Antwort-Datensätzen und werden nicht eigens für das Experiment konstruiert. Dadurch bleibt die Messung reproduzierbar und mit publizierten Ergebnissen vergleichbar. Der System-Prompt ist die einzige manipulierte Komponente. Sprachmodell, Embedding-Modell, Vektorindex, Chunking-Strategie und Retrieval-Tiefe werden konstant gehalten, sodass in jeder Bedingung dieselben Passagen den Generator erreichen und jede Abweichung in der Ausgabe allein am Generator entsteht. Generierungen werden wiederholt, um die Stochastizität der Dekodierung zu berücksichtigen.

Die zwölf Prompts wurden über 100 MS-MARCO-Anfragen auf zwei Evidenzstufen mit je drei Wiederholungen ausgeführt, was 6.768 Antworten ergab. Die Umformulierung veränderte das Verhalten des Systems. Bei der Instruktionsbefolgung schließen elf von zwölf Schätzungen nach Korrektur für multiples Testen die Null aus, zehn davon auf beiden Evidenzstufen, und der größte Effekt beträgt 46 Prozentpunkte. Die Richtung ist dabei nicht festgelegt. Jede Umformulierung der einen Baseline erreichte höhere Werte als deren publizierte Form, jede Umformulierung einer anderen niedrigere. Die Formulierung ist damit ein Parameter, der gut oder schlecht gewählt werden kann, und keine Gefahr, gegen die ein System abgesichert werden müsste. Für die Verankerung in der Evidenz und für die Antwortrelevanz übersteht kein Effekt die Korrektur; beide Maße lagen jedoch in jeder Bedingung nahe am oberen Rand ihres Wertebereichs, sodass diese Untersuchung über sie nichts aussagen kann. Bei der Korrektheit zeigte sich ein Ergebnis über die Messung selbst: Das

vorab festgelegte Maß erkennt keinen Effekt, ein zweites Maß derselben Eigenschaft an denselben Antworten dagegen dreizehn von achtzehn. Beide werden berichtet. Die Hypothese, dass Prompts mit gleichen Anforderungen in unterschiedlicher Formulierung dasselbe Verhalten hervorrufen, wird verworfen.

Acknowledgments

I want to express my sincere gratitude to Professor Ramesh for his guidance and mentorship throughout this thesis.

Contents

Acronyms	9
1 Introduction	10
1.1 Background	10
1.2 Motivation and Research Objective	10
1.3 Thesis Structure	11
Notation	11
2 Preliminaries	13
2.1 Path towards RAG	13
2.2 Components of RAG	14
2.2.1 Knowledge Base / Vector Database	14
2.2.2 Retriever	15
2.2.3 Generator	15
2.2.4 System Prompt	16
2.3 Retrieval-Augmented Generation versus Fine-Tuning	17
2.4 Use-Case of RAG	17
2.5 Advanced RAG Architectures	18
3 Literature Review	20
3.1 Prompt Sensitivity and Instruction-Following in Language Models	20
3.2 Retrieval-Augmented Generation: Architecture, Evaluation, and Failure Modes	21
3.3 Adversarial and Competing Instructions in Retrieved Content	22
3.4 Synthesis and Research Gap	23
4 Formal Model of a RAG System	24
4.1 Modelling Objective	24
4.2 Probabilistic Retrieval	24
4.3 Prompt-Conditioned Generation	25
4.4 Marginalizing over Retrieved Documents	25
4.4.1 RAG-Sequence	25
4.4.2 RAG-Token	25
4.5 The System Prompt: Requirements and Wording	26
4.6 The System Prompt as an Intervention	27
4.7 The Causal Effect	28
4.8 Grounding the Knowledge Base in Established Datasets	29
4.9 What Counts as a Good Answer	29
5 Experimental Setup	31
5.1 Research Question and Hypothesis	31
5.2 Overview of the Experiment	31

5.3	System-Prompt Conditions	32
5.3.1	Three Published Baselines	32
5.3.2	Four Wordings of Each Baseline	33
5.4	Variables and Controlled Factors	35
5.5	Knowledge Base and Query Set	37
5.6	Experimental Procedure	38
5.6.1	Stage 1: Retrieval and the Retrieval Gate	38
5.6.2	Stage 2: Generation	39
5.6.3	The Pipeline	39
5.7	Evaluation	40
5.7.1	The Four Measures	40
5.7.2	The Judge	41
5.7.3	Estimating the Effect of a Rewording	42
5.7.4	Deciding the Hypothesis	42
5.8	Limitations	43
6	Results and Discussion	45
6.1	What Was Collected	45
6.2	How the System Behaved Before Any Comparison	45
6.3	The Effect of Rewording	47
6.3.1	Instruction Adherence	47
6.3.2	Faithfulness	47
6.3.3	Answer Relevance	47
6.3.4	Correctness	48
6.4	The Verdict on the Hypothesis	48
6.5	Reading the Adherence Result	49
6.5.1	Baseline B: The Model Cites, But Not in Every Sentence	49
6.5.2	Baseline C: Compliance Falls When the Instruction Stops Being an Instruction	49
6.5.3	The Direction Is Not Fixed	49
6.6	What These Results Do Not Show	50
7	Conclusion	51
7.1	Summary of Findings	51
7.2	Contribution	52
7.3	Future Work	52

Acronyms

AI Artificial Intelligence.

LLM Large Language Model.

RAG Retrieval-Augmented Generation.

1 Introduction

1.1 Background

Retrieval-Augmented Generation (RAG) systems combine information retrieval with language generation. This design addresses a limit of purely parametric large language models (LLMs), whose knowledge is fixed at training time and cannot be updated without retraining or fine-tuning. Lewis et al. (2020) introduced RAG by combining a pretrained retriever with a pretrained sequence-to-sequence generator, so the model can base its output on external and updatable evidence. A RAG system takes the user query, retrieves relevant information from an external knowledge base, a vector database, and gives that information to a language model with instructions for generating the final response.

The system prompt connects the intended system behaviour to the observable output. The retrieved evidence varies with the query and the knowledge base, but the system prompt is written to specify how the model should behave, which instructions and constraints apply, and which output format it should produce. Because it is written in natural language, its effect on behaviour is hard to predict and hard to check in advance. Its influence therefore has to be examined through controlled experiments rather than assumed.

1.2 Motivation and Research Objective

How a RAG system responds to its system prompt matters in practice. Whether the system follows its instructions reliably decides whether its behaviour can be trusted once deployed. The question is then *how systematic changes in the system prompt can be shown to alter the behaviour and outcomes of a RAG system*.

Isolating that effect can be difficult. The influence of the system prompt overlaps with the influence of the retrieved evidence, of the language model itself, and of the exact wording of the instructions, and all of these have to be controlled before any change in behaviour can be assigned to the prompt. Small, seemingly harmless changes to the wording of an instruction can cause large changes in model behaviour (Salinas and Morstatter, 2024).

The central research question is the following: *To what extent do systematic changes in the system prompt influence the behaviour and outcomes of a RAG system when the user input, knowledge base, retrieval mechanism, and underlying language model are held fixed?* Answering this question requires a formal model of the RAG system in order to identify system prompt as a controlled parameter, as well as an experimental design that holds every other source of variation fixed. The rest of this thesis builds both, and builds them for the simplest retrieval-augmented architecture. In that architecture the system prompt reaches the answer through a single path, which is what makes its effect identifiable.

1.3 Thesis Structure

This thesis has seven chapters. Chapter 2 covers the background needed for the rest of the thesis. It elaborates how retrieval-augmented approaches to language generation developed and introduces the different components of a RAG system. Chapter 3 reviews the relevant literature and identifies the research gap. Chapter 4 builds the formal model of the RAG system used in this thesis. It takes the probabilistic formulation of Lewis et al. (2020) and extends it such that the system prompt becomes an explicit variable of the generator, grounds the knowledge base, and defines what counts as a good answer. Chapter 5 describes the experimental setup. Chapter 6 presents and discusses the results. Chapter 7 concludes the thesis by summarising the findings and outlining directions for future work. The Notation section at the end of this chapter lists every symbol used later, so that the formal chapters can be read without searching back for a definition.

Notation

The notation follows Lewis et al. (2020) for the RAG system and the intervention calculus of Pearl (2009) for the causal statements. Every symbol used in the later chapters is listed here.

The system and its components

- $q \in \mathcal{Q}$ denotes a user query, drawn from the space of possible queries $\mathcal{Q}$, and $P_{\mathcal{Q}}$ denotes the distribution over that space from which queries are drawn.
- $D = \{d_1, d_2, \dots, d_N\} \in \mathcal{D}$ denotes the knowledge base, where each d_i is a document, N is the number of documents, and $\mathcal{D}$ is the space of possible knowledge bases.
- $S \in \mathcal{S}$ denotes a system prompt, drawn from the space of possible system prompts $\mathcal{S}$. It is the standing instruction supplied in the system role, fixed across requests, and is distinct from the query q , which the user supplies anew with each request.
- $C \in \mathcal{C}$ denotes the retrieved context supplied to the language model.
- $y = (y_1, \dots, y_L) \in \mathcal{Y}$ denotes the generated system output, a sequence of L tokens drawn from the space of possible responses $\mathcal{Y}$; $y_{1:i-1}$ denotes the tokens produced before position i .
- R and G denote the retrieval and generation functions of the pipeline, and χ the chunking strategy by which documents are divided into retrievable units.
- $v \in \mathcal{V}$ denotes a token of the vocabulary $\mathcal{V}$, u_v the score the generator assigns it, and $T > 0$ the decoding temperature.

The probabilistic model

- $z \in D$ denotes a single retrieved document, and $\mathcal{Z}_K(q, D)$ the K documents with the highest retrieval probability.
- $p_{\eta}(z \mid q, D)$ denotes the retrieval distribution with parameters η , and $p_{\theta}(y_i \mid q, z, y_{1:i-1}, S)$ the generator with parameters θ .
- $p_{\text{RAG}}(y \mid q, D, S)$ denotes the distribution over outputs obtained by marginalising the generator over the retrieved documents.

- Y denotes the output viewed as a random variable, $M(Y)$ a measured property of it, and ε the randomness introduced by decoding.

The system prompt and its variation

- $\Sigma = \{\sigma_1, \dots, \sigma_m\}$ denotes the set of requirements a system prompt states, where each σ_k is a property an answer either has or does not have, so that $\sigma_k(y) \in \{0, 1\}$ is checkable on any output.
- φ denotes the wording in which those requirements are expressed, and $S = \rho(\Sigma, \varphi)$ denotes the resulting system prompt.
- $S_{b,0}, \dots, S_{b,3}$ denote the four prompt conditions of baseline $b \in \{A, B, C\}$, twelve in all, with $S_{b,0}$ the canonical form of that baseline. All four share the requirement set Σ_b and differ in wording only.

Causal intervention

- $\text{do}(S = S_j)$ denotes the intervention that sets the system prompt to condition j , as distinct from observing it to take that value.
- τ_j denotes the average causal effect of condition S_j relative to the baseline S_0 , taken over the query distribution P_Q , and $\hat{\tau}_j$ its estimate from the sample.
- $\mathbb{E}[\cdot]$ denotes expectation, and $\hat{\cdot}$ marks a quantity estimated from the sample rather than defined in the model.

Evidence and evaluation

- Φ denotes the facts the dataset annotations hold true, Φ_y the facts an answer asserts, Φ_C the facts contained in the retrieved context, and a_q the reference answer annotated for query q .
- $\mathcal{Y}^*(q, C)$ denotes the set of acceptable answers.

Experimental design

- c_{ik} denotes the k -th context given to query q_i , with c^{gold} and c^{ret} the two evidence levels: the annotated supporting passages, and the genuine retriever output.
- $\mathcal{Q}_{\text{direct}}$ and $\mathcal{Q}_{\text{multi-hop}}$ denote the two query categories, and N_Q the size of the query set.
- $\hat{\tau}_{jk}$ denotes the estimate of τ_j within evidence level k .
- The indices are $i \in \{1, \dots, N_Q\}$ over queries, $j \in \{0, \dots, 3\}$ over prompt conditions, $k \in \{1, 2\}$ over evidence levels, and $r \in \{1, \dots, n\}$ over the n repetitions of each cell.

2 Preliminaries

Chapter 1 established that the system prompt is the interface through which a RAG system’s intended behaviour is specified, and asked how systematically varying that interface changes system behaviour and outcomes. Answering that question formally first requires the components of a RAG system and the reasons behind its design. Section 2.1 traces how retrieval-augmented approaches to language generation developed. Section 2.2 introduces the four components used throughout the later chapters. Section 2.3 contrasts retrieval with fine-tuning, Section 2.4 gives the settings in which RAG is typically deployed, and Section 2.5 reviews the advanced architectures this thesis deliberately does not use.

2.1 Path towards RAG

The need to store knowledge outside of any single memory, and to retrieve it again on demand, predates computing. Inca administrators recorded census and tax data in knotted cords, quipu, that only specially trained officials could encode or decode. Long before that, the Library of Alexandria’s cataloguer Callimachus organised its holdings by author and subject in the Pinakes. This was one of the first attempts to make a large external collection searchable rather than merely stored. Bush (1945) argued the need for machine-assisted retrieval and proposed a device that could store a person’s library and retrieve from it through associative links between related items instead of a fixed order. Codd (1970) gave the idea a rigorous computational form by treating a knowledge store as a formally queryable structure. Salton et al. (1975) then extended retrieval from exact structured queries to similarity ranking over unstructured text. This is the direct ancestor of the embedding-based retrieval described later in this chapter.

The need to retrieve from an external store instead of remembering everything internally led to the first question-answering systems. Early systems relied on hand-crafted rules and pattern matching over the input question, and mapped recognised syntactic patterns onto retrieval or answer-generation procedures (Pandya and Bhatt, 2021). Statistical and later neural methods replaced those rule-based components. The dominant approach shifted toward learning the mapping from question to answer from data. Large language models are the end point of that shift since they generate answers from parameters learned over large text corpora rather than from any explicit rule set.

Large language models generate text using only parametric knowledge, that is, information encoded in their weights during training (Wang et al., 2024). This leads to two problems. First, a model’s knowledge is fixed at the point its training data was collected. Any event, fact, or document after that point is unknown to the model unless it is retrained or fine-tuned. Second, purely parametric models are prone to hallucination, because they can generate fluent, plausible-sounding statements that no evidence supports, since nothing in their design forces an output to trace back to a verifiable source.

Both problems are the reason to give a language model access to an external and updatable source of evidence at inference time, instead of relying only on what is stored in its parameters (Lin et al.,

2025). Lewis et al. (2020) formalised this idea as Retrieval-Augmented Generation. RAG combines a retriever, which selects relevant passages from a document collection, with a pretrained generator, which conditions its output on those retrieved passages in addition to the input query.

RAG has developed beyond that original design since it was introduced. Gao et al. (2023c) describe three stages: Naive RAG, which runs retrieval and generation as one fixed pipeline; Advanced RAG, which adds steps such as query reformulation or re-ranking of retrieved passages before generation; and Modular RAG, which makes retrieval, generation, and other steps separate components that can be rearranged for different tasks. Retrieval and generation stay separate steps in all three. This thesis therefore treats them as two distinct functions, such that their interaction with the system prompt can be studied on its own, as will be formalised in Chapter 4.

2.2 Components of RAG

A RAG system has four components: the knowledge base the evidence comes from, the retriever that selects relevant evidence for a given query, the generator that produces a response from the query and the retrieved evidence, and the system prompt that specifies how the generator should behave. This section introduces each component. Chapter 4 treats them formally. Figure 2.1 shows how they fit together and which data passes between them.

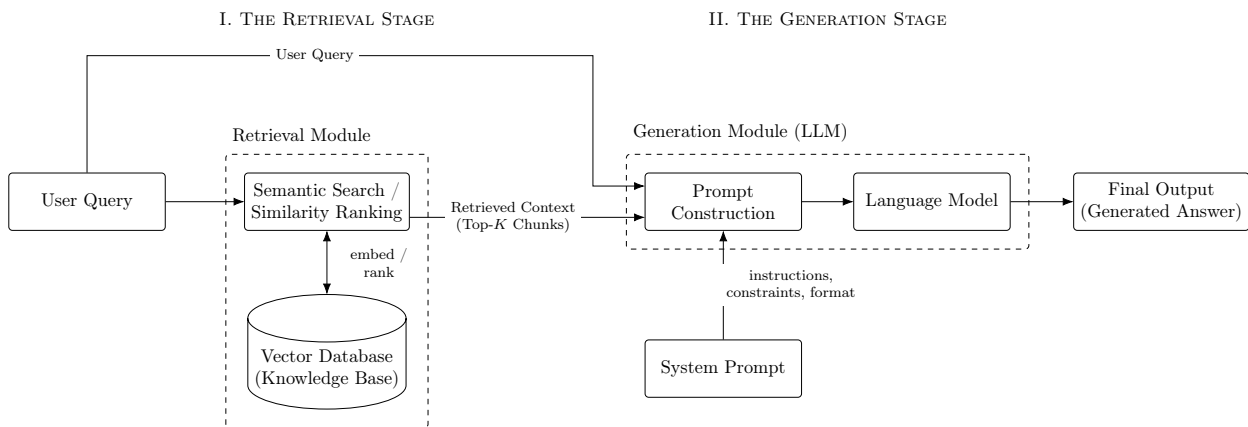


Figure 2.1: Architecture of a RAG system, showing the four components introduced. The retrieval stage matches the user query against a vector database and returns the top- K chunks. The generation stage combines those chunks with the user query and the system prompt into a single prompt and produces the answer.

2.2.1 Knowledge Base / Vector Database

The knowledge base $D = \{d_1, d_2, \dots, d_N\}$ is the collection of documents the evidence comes from. In most RAG systems, documents are first split into smaller passages, called chunks. Chunking is needed because a language model can only take in a limited amount of context.

An embedding model maps each chunk to a high-dimensional vector, so that chunks with similar meaning end up close together in the embedding space. Representing text as vectors and measuring relatedness by geometric proximity goes back to the vector space model of Salton et al. (1975). Similarity between a query vector and a chunk vector is usually measured by the cosine of the angle between them.

The embeddings are stored in a vector database. This database supports fast nearest-neighbour search over large collections of chunks. Comparing a query against every stored vector is exact but becomes too slow as the collection grows. Therefore vector databases index the vectors and use

approximate nearest-neighbour search. The most widely used index of this kind is the hierarchical navigable small world graph of Malkov and Yashunin (2020).

Several systems build on such indexes. Pan et al. (2024) survey how they differ in indexing, query processing and storage. What matters here is how they are deployed. FAISS is a library embedded directly in an application, with fine-grained control over the index structure (Johnson et al., 2021). Chroma is a lightweight database aimed at local development and small deployments. The `pgvector` extension adds vector columns and similarity operators to PostgreSQL, so that vectors can sit next to existing relational data. Pinecone is a managed service in which the provider handles indexing and scaling.

2.2.2 Retriever

Given a user query q and a knowledge base D , the retriever selects the chunks that are relevant for answering q . Chapter 4 formalises this as a probability distribution $p_\eta(z \mid q, D)$ over relevant documents. Two approaches are common, and they match a query to a passage in different ways.

Dense Retrieval

Dense retrieval embeds the query with the same embedding model used for the knowledge base and ranks chunks by the similarity between the query vector and each chunk vector. Because the embedding is learned, two passages that express the same content in different words end up close together. Hence, a query can match a chunk with which it shares no vocabulary. This is the mechanism the original RAG architecture of Lewis et al. (2020) relies on.

Sparse Retrieval

Sparse retrieval scores chunks by term overlap with the query. The standard method is BM25 (Robertson and Zaragoza, 2009). BM25 weights each matching term by how often it occurs in the chunk, how rare it is across the collection, and how long the chunk is. Common words therefore contribute little, and long chunks are not favoured only for containing more words. The representation is sparse since a chunk is described by the vocabulary it contains. BM25 requires no training and remains a strong baseline.

Retrieval Depth

Whichever method is used, the top- K ranked chunks form the retrieved context. Retrieval quality depends on both how many chunks are retrieved and how precise the ranking is. Too few chunks risk missing needed evidence, while too many risk adding irrelevant information to the generation step. The order of the chunks also matters. Liu et al. (2024) report that answer accuracy depends on where the relevant passage sits within the context. This thesis fixes the retriever, the chunking and K across all experimental conditions. Therefore, changes in the output cannot be traced back to retrieval.

2.2.3 Generator

The generator is the large language model that produces the final response. Such a model is a neural network trained on a large text corpus to predict the next token in a sequence. It generates text by autoregressive decoding. Its parameters, written θ in Chapter 4, encode what the model learned during training. This encoded knowledge is often called parametric memory.

Within a RAG pipeline the model reads the retrieved chunks, selects the parts that answer the query, resolves references between them, and writes the result as fluent text (Lewis et al., 2020).

In practice the query, the retrieved context and the system prompt are not passed to the model separately. A fixed template combines them into a single input sequence with the system prompt first so that it shapes how everything after it is read. Chapter 4 formalises the generator as $p_\theta(y_i | q, z, y_{1:i-1}, S)$, the probability of the next token given the query, a retrieved document, the tokens already produced and the system prompt. Because these three together determine the model’s input, changing any one of them can change the output. The others must therefore be held fixed if a change in behaviour is to be attributed to the system prompt alone.

Decoding and the Temperature Parameter

Generation is not deterministic by default. At each step the model produces a score u_v for every token v in the vocabulary $\mathcal{V}$, and the softmax function, with a temperature parameter $T > 0$, turns these scores into a probability distribution:

$$p(v) = \frac{\exp(u_v/T)}{\sum_{v' \in \mathcal{V}} \exp(u_{v'}/T)}. \quad (2.1)$$

The temperature rescales the scores before they are normalised, and it controls how much probability mass is left for tokens the model considers less likely (Holtzman et al., 2020). As T approaches zero the distribution concentrates on the highest-scoring token, and decoding becomes effectively deterministic. As T grows the distribution flattens, lower-scoring tokens become reachable, and the output becomes more varied across runs.

For open-ended writing this variety is useful. For a RAG system it is mostly harmful. A token that the model scores as unlikely given the retrieved context is one that the evidence does not support well. Consequently, raising the temperature increases the chance of producing a statement the retrieved documents do not contain. High temperature also makes the system harder to evaluate, since the same query and the same evidence no longer yield the same answer. RAG systems intended to be faithful to their sources are therefore usually run at a low temperature. This thesis holds T fixed across all experimental conditions, but fixes it at the API default of 1.0 rather than at a low value. The reason is that the object of study is the difference between two prompt conditions, not the best attainable answer quality, and a fixed temperature gives every condition the same amount of decoding noise. The noise that remains at that setting is the reason each condition is repeated several times in Chapter 5, and its cost is stated in Section 5.8.

2.2.4 System Prompt

The system prompt S specifies the instructions, constraints and output requirements the generator should follow. The query changes with every request, and the retrieved context changes with the query and the knowledge base. The system prompt is usually fixed across many requests.

The system prompt has two functions. The first is to define the role it occupies, the audience it addresses, and the format its output takes. The second is to install hard constraints on that behaviour, of which the grounding constraint is the most consequential. A grounding constraint restricts the model to the retrieved context and specifies what to do when that context is inadequate. Listing 2.1 illustrates both in a minimal example.

You are a customer service assistant for a parcel delivery company.


```

GROUNDING RULE
Answer only from the context provided below. Do not use information
from your training data, even where you are confident that it is
correct.

INSUFFICIENT EVIDENCE
If the context does not contain the answer, reply with exactly:
The provided documents do not contain this information.

FORMAT
Answer in at most three sentences. Cite the identifier of every
document you use, in square brackets.

```

Listing 2.1: A minimal system prompt combining an operational scope with two hard constraints.

The first line fixes the operational scope. The two blocks that follow are hard constraints. One restricts the admissible source of fact, the other defines the response for the case in which no admissible source exists. The final block constrains form rather than content, and belongs to the first function. Each of these instructions is stated so that it can be checked against an output. That is the property Chapter 4 requires of the requirement set Σ .

2.3 Retrieval-Augmented Generation versus Fine-Tuning

Fine-tuning is the other established way of adapting a general-purpose language model to a specific application. It is worth stating how it differs from retrieval before Section 2.4 turns to use cases. Fine-tuning continues training on domain-specific data and changes the model parameters θ , consequently what the model learns enters the parametric memory described in Section 2.2.3. RAG leaves θ untouched and changes what is placed in the context window at inference time. The two therefore act on different parts of the system rather than competing to answer one question. Fine-tuning requires a training run over the model’s parameters whenever the underlying information changes. Adding a document to a knowledge base requires only that the new chunks be embedded and indexed (Cheerla, 2025). Wang et al. (2024) survey the difficulty of locating and editing individual facts inside model parameters. This matters wherever information is corrected or retracted, or must be deleted on request. It is also one reason retrieval is preferred in regulated settings, where sensitive data can remain in a controlled external store instead of entering the parameters (He et al., 2025). Because a RAG answer is produced from a specific set of retrieved chunks, each claim in the output can in principle be checked against the passage that supports it, and unsupported claims can be identified at the level of the individual statement (Niu et al., 2024).

2.4 Use-Case of RAG

RAG systems are typically used where the needed knowledge is too large, too specific, or changes too often to be captured reliably by a model’s parametric knowledge. Deployments span many domains: question-answering over enterprise and internal document collections (Cheerla, 2025), customer support systems that must answer from current product or policy documentation, coding assistants that retrieve from a specific codebase instead of relying on general programming knowledge, legal question-answering over case law and contract repositories (Pipitone and Alami, 2024), and clinical decision support that grounds its answers in current medical literature and institutional protocols (He et al., 2025).

2.5 Advanced RAG Architectures

The architecture described so far retrieves once per query and generates once from what was retrieved. Recent work relaxes both assumptions, and the resulting variants can be grouped by which stage of the pipeline they modify. The summary below is given for completeness. None of these variants is used here, because this thesis studies the standard architecture.

Conversational RAG. Retaining prior turns of a dialogue as state, so that a follow-up question can be resolved against what was already asked and answered. A query such as “and the second one?” is meaningless to a retriever on its own, so the conversation history has to supply what is missing.

Pre-retrieval optimisation. Rewriting or expanding the query before it reaches the retriever. Hypothetical Document Embeddings (HyDE) generate a hypothetical answer to the query and embed that instead of the query itself, on the argument that an answer lies closer in embedding space to the passage containing it than a question does (Gao et al., 2023a).

Routing and adaptive retrieval. Deciding where and how much to retrieve. Where several collections or indexes are available, a routing step can select among them according to what the query is about, rather than searching one combined collection for everything. Adaptive-RAG goes further and varies the depth of retrieval with the difficulty of the query, answering simple questions directly and reserving multi-step retrieval for those that need it (Jeong et al., 2024).

Self-correction and reflection. Evaluating retrieved evidence before or during generation. Corrective RAG (CRAG) grades each retrieved document, decomposes the ones it keeps into smaller knowledge strips, discards the strips that are not relevant, and falls back to a wider search when retrieval fails (Yan et al., 2024). Self-RAG instead trains the model to decide for itself when to retrieve and to critique its own draft output, so that retrieval and generation interleave rather than run once each (Asai et al., 2024).

Agentic RAG. Treating retrieval as one action among several available to an agent. A meta-agent delegates to document-specific or tool-specific agents and combines their results, which turns a fixed pipeline into a workflow whose steps are chosen at run time (Singh et al., 2025).

Understanding the standard architecture formalised in this thesis remains a prerequisite for all advanced RAG architectures. Each variant changes the retrieval function R , the generation function G , or the number of times they are applied, but all of them are still built from a retrieval step and a generation step, and all of them still produce a distribution over outputs. What changes is which variables that distribution depends on, and, as the next paragraph shows, that change is exactly what makes the effect of the system prompt harder to identify.

This thesis therefore implements the simplest architecture and none of the variants above. The reason is methodological as the research objective is to estimate the causal effect of the system prompt on the output, that Chapter 4 formalises as the intervention $\text{do}(S = S_j)$. This particular effect is identifiable only as long as the system prompt reaches the answer through a single path. Each of the advanced variants breaks that condition. Under Self-RAG the model decides for itself when to retrieve (Asai et al., 2024), thus the system prompt influences which documents are fetched as well as how they are used. Under routing, corrective and agentic designs the prompt shapes the decisions of the router, the retrieval evaluator or the meta-agent, so that the retrieved context C ceases to be determined by the query and the corpus alone. Retrieval would then have to be written

$p_\eta(z \mid q, D, S)$ rather than $p_\eta(z \mid q, D)$, and the arrow from S into the retriever that Figure 4.1 deliberately omits would be present. A measured difference between two prompt conditions could be attributed either to the prompt’s effect on generation or to its effect on what was retrieved, with no way to separate the two.

3 Literature Review

Chapter 2 introduced the components of a RAG system. This chapter reviews three areas of literature relevant to studying how the system prompt affects such a system. Section 3.1 covers prompt sensitivity and instruction-following in language models generally. Section 3.2 turns to retrieval-augmented generation, its architecture, evaluation, and known failure modes, and Section 3.3 examines adversarial and competing instructions delivered through retrieved content. Section 3.4 brings the three together into the research gap this thesis addresses.

3.1 Prompt Sensitivity and Instruction-Following in Language Models

The first branch of literature studies language models with no retrieval component at all. Cheng and Mastropaolo (2026) vary the system prompt given to instruction-tuned models for code generation, crossing prompt specificity and prompting strategy with model, programming language and decoding settings. They report that the system prompt moves the quality of generated code independently of the task description. Theirs is the closest published design to the one used here, and the present experiment carries it from code generation into a retrieval-augmented setting, where the model is also given evidence to work from. Mu et al. (2025) examine how reliably models honour a system prompt when the user turn contains instructions conflicting with it. Their result establishes the system prompt as a control whose authority is contingent rather than guaranteed. Zheng et al. (2024) assign personas through the system prompt across a large set of roles and find that doing so does not reliably improve accuracy over one carrying no persona at all.

A larger group varies the user-side prompt. Lu et al. (2024) compare prompting techniques for in-context learning and measure sensitivity through gradient-based saliency scores. They report that sensitivity correlates negatively with accuracy and can serve as an unsupervised proxy for it. Zhuo et al. (2024) propose ProSA, which scores instance-level variation in the task instruction against decoding confidence, and report that larger models are more robust to such variation and that few-shot examples reduce it. Salinas and Morstatter (2024) perturb task prompts across text classification tasks (adding a trailing space, requesting the output in XML, or prefixing a jailbreak) and find that perturbations of this size change the labels the model assigns. Sclar et al. (2024) vary only the formatting of few-shot task templates (separators, casing and spacing, none of which carries propositional content) and report differences of up to 76 accuracy points between formats a reader would judge equivalent. Hua et al. (2025) run twelve task templates across several benchmarks and argue that much of the measured sensitivity is produced by the evaluation method (log-likelihood scoring and rigid answer matching) rather than by the model, since the variance falls once an LLM judge is used instead. Qin et al. (2026) decompose a model’s output score into interactions between input variables and observe that small changes to the prompt destabilise those interactions even where the final output is unchanged.

Two further studies sit outside both groups. Ruiz and Cardona-Betancourt (2026) state a rule in the instruction given to a model and then shift the contingencies that rule was written for, and

find that models keep following the stated rule after it has stopped paying, a pattern that resembles rule-governed behaviour in humans. The channel carrying the rule is not the variable of interest there, but the finding bears on how a stated requirement competes with what the evidence in front of the model supports. Ying et al. (2024) study conflict between a model’s parametric memory and the evidence supplied in its context, and classify models as intuitive or dependent by which source they favour. That conflict runs between memory and context, not between a system prompt and a user instruction, so the study bears on the evidence axis of this thesis rather than on its prompt axis.

Two conclusions follow. The first is that the system prompt is a behavioural control variable whose effect can be isolated and measured given a properly controlled setup, which the three system-prompt studies support directly (Cheng and Mastropaolo, 2026; Mu et al., 2025; Zheng et al., 2024). The second is that instructions are sensitive to changes that preserve their meaning, whether lexical (Lu et al., 2024; Zhuo et al., 2024; Salinas and Morstatter, 2024), purely formatting-level (Sclar et al., 2024), or not visible in the output at all (Qin et al., 2026), and that the apparent size of such an effect depends on how it is measured (Hua et al., 2025). This second conclusion is established for the user-side prompt, and its transfer to the system prompt is an assumption rather than a result. The assumption is reasonable, since both texts reach the same decoder as tokens in the same context window, but it is untested.

3.2 Retrieval-Augmented Generation: Architecture, Evaluation, and Failure Modes

The second branch narrows to retrieval-augmented generation itself, and to the failure modes it shows even when no one is manipulating the system prompt.

Lewis et al. (2020) introduce Retrieval-Augmented Generation, combining a pretrained neural retriever with a pretrained sequence-to-sequence generator, and show that conditioning generation on retrieved evidence outperforms purely parametric models on knowledge-intensive tasks. Their probabilistic formulation, in which retrieval is a distribution over documents and the retrieved document is marginalised out, is the direct basis for the model formalised in Chapter 4. Gao et al. (2023c) survey the development of RAG systems through Naive, Advanced and Modular paradigms, organising the field around a tripartite retrieval-generation-augmentation structure. Their taxonomy supports treating retrieval and generation as separable functions whose interaction with the system prompt can be studied independently, as this thesis does. Two later developments extend this taxonomy in directions this thesis deliberately does not follow. Edge et al. (2024) restructure the retrieval side into a knowledge graph, grouped into thematic communities and pre-summarised by an LLM, so that a query is answered by traversing entities and community summaries rather than by a flat nearest-neighbour lookup over D . Singh et al. (2025) and Jin et al. (2025) restructure the generation side into an agent loop, in which the model itself decides when and what to retrieve, calling R repeatedly and adaptively rather than once per query; Jin et al. (2025) train this behaviour directly with reinforcement learning and report large gains over standard RAG baselines. Both directions relax exactly the components Chapter 4 holds fixed: R in the first case, and the single-call structure of G in the second. For that reason this thesis retains Gao et al.’s simplest, Naive-RAG form since any change in the outcomes measured here can then be attributed to the system prompt, rather than to the added architectural complexity of a graph-structured retriever or an agentic generation loop.

Es et al. (2023) propose Ragas, a reference-free evaluation framework for RAG pipelines built around three metrics that need no ground-truth human annotation: faithfulness, answer relevance and context relevance. This thesis adopts two of them directly as measures, faithfulness and answer relevance. The third, context relevance, is a retrieval-side measure and is excluded for the reasons given in Section 5.7. Their framework also motivates evaluating the effect of the system prompt independently of retrieval quality. Chen et al. (2024) construct the Retrieval-Augmented Generation Benchmark (RGB) and show that current language models tolerate a moderate amount of retrieval noise but struggle with negative rejection, with information integration across multiple retrieved passages, and with robustness to counterfactual or contradictory evidence. Two of their four testbeds shape the design of this thesis. Noise robustness and information integration become the two query categories of Section 5.5, and their finding on retrieval noise motivates comparing an idealised evidence level with the retriever’s real output. The other two testbeds, negative rejection and counterfactual robustness, ask about a system’s response to missing or false evidence rather than about the wording of its instructions, and they are not implemented here. Niu et al. (2024) introduce RAGTruth, a manually annotated corpus of word-level hallucinations produced by RAG systems, and benchmark hallucination-detection methods across multiple language models. Their finding that hallucination persists even when relevant evidence has been retrieved motivates measuring faithfulness as an outcome distinct from retrieval quality, as this thesis does. Yu et al. (2024) draw this body of work together into a survey of RAG evaluation, separating the targets that assess the retrieval stage from those that assess generation and naming three that belong to the latter: relevance, faithfulness and correctness. The measures adopted in Section 5.7 come from that account, and this thesis’s restriction to generation-stage measurement rests on the separation it draws.

Together, these studies establish the architecture, evaluation vocabulary, and known failure modes of RAG systems specifically. They show that a RAG system’s output quality depends on both retrieval quality and how faithfully the generator uses the retrieved evidence (Es et al., 2023; Niu et al., 2024). Current systems already struggle with conflicting or insufficient retrieved evidence, even without any prompt manipulation (Chen et al., 2024). None of the studies in this branch, however, treats the system prompt as a controlled experimental variable. It is either fixed across conditions or not specified at all.

3.3 Adversarial and Competing Instructions in Retrieved Content

A third line of work concerns instructions that arrive in the retrieved content rather than in the system prompt, and that compete with the system prompt for control of the model’s behaviour. The first branch shows that a model’s own system prompt can lose authority to instructions arriving in the user turn (Mu et al., 2025); Greshake et al. (2023) go further, showing that text hidden inside a document the model is told to process can override the system prompt altogether. This is the same instruction competition taken to its adversarial extreme, except that the competing instruction reaches the model through the retrieved context C rather than through S , which this thesis holds fixed rather than varies. Zhang et al. (2025) build a RAG-specific benchmark on this basis, covering 13 poisoning attack methods and 7 defenses across 5 base question-answering datasets and 10 expanded variants. Chen et al. (2024) and Niu et al. (2024) in the second branch already find that retrieved evidence is often noisy, incomplete, or contradictory, so competing content reaches the generator even when no attacker is involved. The work in this branch, however, treats instruction competition exclusively as a security problem: an adversary deliberately crafts malicious content,

and the goal is to detect or defend against it. Neither examines benign linguistic variation in the system prompt itself, the kind the prompt-sensitivity literature reviewed above studies outside of RAG, and the kind this thesis’s conditions are built to isolate inside it. The question here is therefore not whether a RAG system can be attacked, but whether ordinary, well-intentioned rewordings of its own instructions change its behaviour at all.

3.4 Synthesis and Research Gap

The first branch shows two things separately: that the system prompt is a controllable way of shaping language model behaviour, and that instructions are sensitive to rewording that leaves their semantic meaning intact. It establishes the controllability result for the system prompt, the sensitivity result mostly for the user-side prompt, and both outside any retrieval-augmented setting. The second branch establishes the architecture, evaluation vocabulary, and known failure modes of RAG systems, including problems with evidence integration, rejection, and faithfulness, but does not treat the system prompt as a controlled experimental variable. The third branch shows that instructions can compete with the system prompt in RAG systems and that this competition is actively studied, but only as a deliberate security threat delivered through retrieved content, never as benign, controlled linguistic variation of the system prompt itself. The overlap between all three branches is still largely unexamined. Ying et al. (2024) comes nearest, but the conflict studied there runs between a model’s parametric memory and its retrieved context rather than between two wordings of one instruction, and neither retrieval nor the underlying model is held fixed as an explicit experimental control.

This thesis addresses that gap. It treats the system prompt as a controlled experimental variable inside a RAG pipeline, holding the query, knowledge base, retrieval mechanism, and underlying language model fixed, so that any change in the system’s behaviour and outcomes can be traced to the prompt itself. Every condition varies only the wording of the instructions and leaves what they require untouched, which separates the effect of phrasing from the effect of content.

The next Chapter, Chapter 4, builds the formal model that makes this control precise. It describes the RAG system as a retrieval function combined with a generation function, with the system prompt as an explicit parameter of the generation function, and states the assignment of a prompt condition as an intervention on that model.

4 Formal Model of a RAG System

Chapter 3 identified the research gap: prompt sensitivity is studied outside retrieval, and RAG evaluation does not treat the system prompt as a controlled variable. Closing it requires a model of the system precise enough to isolate the system prompt. This chapter builds that model on the probabilistic formulation of retrieval-augmented generation given by Lewis et al. (2020), and extends it by making the system prompt an explicit variable of the generation process.

4.1 Modelling Objective

The formalisation does two things. It states the probabilistic relationship between the user query, the retrieved documents and the generated output. It also provides the basis for treating the system prompt as an experimental intervention whose effect can be measured while the query, the knowledge base, the retriever and the language model are held constant.

The model can be read as a chain,

$$(q, D) \longrightarrow p_\eta(z \mid q, D) \longrightarrow p_\theta(y_i \mid q, z, y_{1:i-1}, S) \longrightarrow p_{\text{RAG}}(y \mid q, D, S),$$

in which the system prompt S enters at the generation stage only. The research question of this thesis concerns that final distribution: what changes in $p_{\text{RAG}}(y \mid q, D, S)$ when S is changed?

The symbols used below are those listed in the Notation section of Chapter 1. The one departure from Lewis et al. (2020) is that they write the input sequence as x , whereas this thesis writes it as q , since the input is always a user query. Their notation is otherwise kept unchanged.

4.2 Probabilistic Retrieval

Retrieval is represented as a probability distribution over documents rather than as a single choice:

$$p_\eta(z \mid q, D). \tag{4.1}$$

Equation (4.1) reads as the probability the retriever assigns to document z , given the query q and the knowledge base D . Writing retrieval this way means the retriever never has to commit to one document being the correct one. It spreads probability over the candidates, and documents it considers more relevant receive more of it.

A knowledge base may hold far more documents than can be evaluated in practice. The formulation therefore keeps only the K documents with the highest retrieval probability,

$$\mathcal{Z}_K(q, D) = \underset{z \in D}{\text{TopK}} \ p_\eta(z \mid q, D), \tag{4.2}$$

and the retrieved context handed to the generator is the resulting collection,

$$C = \{z_1, \dots, z_K\}, \quad z_k \in \mathcal{Z}_K(q, D). \tag{4.3}$$

The distinction between z and C matters later: the first is a candidate the model sums over, the second the concrete collection the running system retrieves and logs.

4.3 Prompt-Conditioned Generation

Lewis et al. (2020) write the generator at the level of a single token as $p_\theta(y_i \mid q, z, y_{1:i-1})$: the probability of the next token given the query, one retrieved document, and the tokens already produced. This thesis adds the system prompt as a further conditioning variable:

$$p_\theta(y_i \mid q, z, y_{1:i-1}, S). \quad (4.4)$$

Equation (4.4) is the point at which this thesis departs from the original formulation, and the departure is deliberately small. The generator is the same model with the same parameters θ ; what changes is that the system prompt is now written into the conditioning information rather than left implicit.

Because generation is autoregressive, the probability of a complete output sequence follows from the token probabilities by multiplication:

$$p_\theta(y \mid q, z, S) = \prod_{i=1}^L p_\theta(y_i \mid q, z, y_{1:i-1}, S). \quad (4.5)$$

4.4 Marginalizing over Retrieved Documents

Equation (4.5) gives the probability of an output under one particular document z . Since the system does not know which retrieved document supports the answer, the model averages over the candidates, weighting each by its retrieval probability. Lewis et al. (2020) give two ways of doing this, which differ only in where the sum is placed.

4.4.1 RAG-Sequence

RAG-Sequence assumes that a single retrieved document is responsible for the whole answer. The probability of an output is the weighted average, over documents, of the probability that document would produce that entire sequence:

$$p_{\text{RAG-Sequence}}(y \mid q, D, S) \approx \sum_{z \in \mathcal{Z}_K(q, D)} p_\eta(z \mid q, D) \prod_{i=1}^L p_\theta(y_i \mid q, z, y_{1:i-1}, S). \quad (4.6)$$

The sum stands outside the product, and that is what encodes the assumption: one document is fixed, the entire sequence is generated from it, and only then is the next document considered.

4.4.2 RAG-Token

RAG-Token makes the weaker assumption that different parts of an answer may come from different documents. The average over documents is taken separately at every token:

$$p_{\text{RAG-Token}}(y \mid q, D, S) \approx \prod_{i=1}^L \sum_{z \in \mathcal{Z}_K(q, D)} p_\eta(z \mid q, D) p_\theta(y_i \mid q, z, y_{1:i-1}, S). \quad (4.7)$$

The whole difference between the two is the order of the two operators: $\sum_z \prod_i$ against $\prod_i \sum_z$. RAG-Sequence commits to one document for the length of the answer; RAG-Token lets one docu-

ment supply the first half of a sentence and another the second, the more flexible assumption and the one better suited to answers that combine several sources.

Where the distinction does not matter, both are written here as $p_{\text{RAG}}(y \mid q, D, S)$. It does not matter for the experiment: the system prompt S appears in exactly the same place in both, inside the generator and nowhere else.

$$p_{\text{RAG}}(y \mid q, D, S). \quad (4.8)$$

4.5 The System Prompt: Requirements and Wording

The system prompt enters Equation (4.4) as a single symbol, but it is a text with internal structure, and the experiment varies that structure deliberately. A prompt states a set of requirements and states them in particular words. Writing the two separately gives

$$S = \rho(\Sigma, \varphi), \quad \Sigma = \{\sigma_1, \dots, \sigma_m\}, \quad (4.9)$$

which separates what a prompt asks for from the words it asks in. Two prompts built from the same Σ ask for the same thing. Two built from the same φ say things the same way.

The conditions cross two factors. The first is the requirement set. Three are used, each taken from a prompt published with a retrieval-augmented system and reproduced in Section 5.3. Writing Σ_A , Σ_B and Σ_C for the three: Σ_A is the response-generation prompt of Friel et al. (2024), which states a single requirement; Σ_B is the VANILLA instruction of Gao et al. (2023b), which states about ten clauses falling into four independently checkable families; and Σ_C is the instruction of Mao et al. (2024), which states three. The three sets therefore differ in how much they ask for, which is what Section 5.3 uses them for.

The second factor is the wording. Four wordings are written for each requirement set, holding Σ fixed and changing only φ . The axes along which the wording is varied are taken from three prior studies rather than devised here. Wahle et al. (2024) vary prompts by linguistically defined paraphrase type rather than by ad hoc rewriting, and group those types into families, of which morphology, syntax and lexicon apply to a standing instruction. Zhan et al. (2024) isolate the lexical axis by substituting single semantically similar words, and report that models are over-sensitive to variations imperceptible to a reader. Kostić et al. (2026) separate a lexical pipeline built on synonym substitution from a syntactic one built on dependency parsing, and find that the two behave differently since lexical perturbation degrades performance consistently, syntactic perturbation does not.

This thesis follows their approach of varying a prompt along linguistically defined, meaning-preserving axes one at a time, and applies it to the standing system prompt of a retrieval-augmented system. Its research question requires that application, and none of the three studies addresses it. Writing $S_{b,j}$ for the j -th wording of baseline b :

$S_{b,0} = \rho(\Sigma_b, \varphi_0)$, **canonical**. The published form, unchanged. The reference the other three are measured against.

$S_{b,1} = \rho(\Sigma_b, \varphi_1)$, **lexical**. Synonym substitution with clause order and voice held fixed.

$S_{b,2} = \rho(\Sigma_b, \varphi_2)$, **syntactic**. Clause reordering and active to passive, with the vocabulary held as far as English allows.

$S_{b,3} = \rho(\Sigma_b, \varphi_3)$, **format**. The same wording laid out as a list rather than as prose.

Twelve conditions follow. Each axis changes one thing and holds the rest fixed. That is what makes a difference between two conditions attributable to that axis, and it is easy to break by accident. The syntactic variant carries obligation uniformly through the construction *is to be*, so that a change of voice does not also change how strongly something is required; the format variant writes its items as bare noun phrases for the same reason. No condition strengthens or weakens an obligation, because that would change Σ rather than φ .

Baseline *A* states the fewest requirements and is short enough to give in full. The four conditions read:

```

A0  Use the following pieces of context to answer the question.

A1  Employ the following excerpts of material to address the query.

A2  The question is to be answered using the context pieces below.

A3  CONTEXT: the following pieces
    TASK: use them to answer the question

```

Listing 4.1: The four conditions of baseline *A*, whose canonical form is the prompt of Friel et al. (2024).

The variants for baselines *B* and *C* are written on the same principle and are longer in proportion to their requirement sets. Within each baseline the four conditions are held to within one tenth of the canonical form in length, so that a difference between two of them cannot be read as an effect of how much text the prompt contains rather than of the axis being varied.

Crossing the wording axes with three requirement sets rather than one is what lets the central question be asked properly. If the system responded only to what a prompt requires and not to how it is written, $S_{b,0}$ through $S_{b,3}$ would produce the same output distribution within every baseline, since they differ in φ alone. Whether they do is the empirical question. The format condition in particular follows Sclar et al. (2024), who report large performance differences from formatting changes of this kind. They apply those changes to few-shot task templates rather than to a system prompt, so carrying the manipulation across to S is itself part of what is tested here. Using three baselines also answers an objection a single one cannot. An effect found with one requirement set might be a quirk of that particular prompt rather than a property of rewording. Three sets, differing in how much they ask for, show whether the same sensitivity appears in more than one place.

Every contrast is taken within a baseline. Comparing $S_{B,1}$ against $S_{A,0}$ would confound a change of wording with a change in what is asked for, so each variant is measured against the canonical form of its own requirement set.

4.6 The System Prompt as an Intervention

The prompt appears in the generator (Equation (4.4)) and not in the retriever (Equation (4.1)). The retrieval term is therefore written without S among its arguments: the system prompt is supplied to the generator but plays no part in choosing documents. In the notation of Pearl (2009), S has an effect on y and no effect on z . Figure 4.1 shows the resulting structure.

For the effect of the prompt to be interpretable, the following are held constant across conditions. Each query is used unchanged in every condition it is run in, so that every comparison is made within a query rather than between queries, and the knowledge base is fixed at $D = D_0$. The

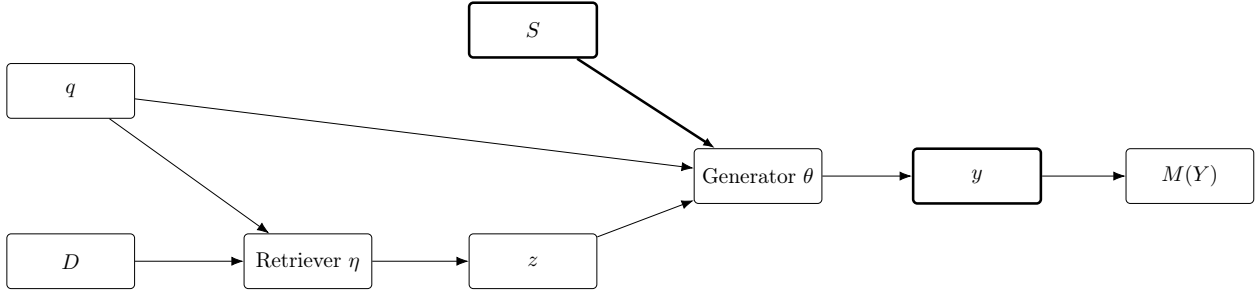


Figure 4.1: The system prompt S and the output y are drawn in bold. S enters the generator and has no arrow into the retriever, which is what allows a change in y to be attributed to a change in S .

retriever and its parameters are fixed at $\eta = \eta_0$, and the language model at $\theta = \theta_0$, so the model is neither retrained nor replaced between conditions. Generation is prompt-dependent: although θ stays fixed, the conditional distribution it defines may differ between S_0 and S_j . Finally, decoding is stochastic, so the same configuration can produce different outputs from one run to the next; the term ε carries that variation:

$$Y = G(q, z, S; \theta, \varepsilon), \quad (4.10)$$

which is why each condition is repeated several times in Chapter 5.

4.7 The Causal Effect

Setting the system prompt to a fixed value is an intervention, written $\text{do}(S = S_j)$ following Pearl (2009). The notation separates *setting* a variable from merely *observing* it. For a query q under prompt condition S_j , the output of the system is the random variable

$$Y \sim p_{\text{RAG}}(y \mid q, D, S_j). \quad (4.11)$$

For one query the effect of the prompt is the difference in the expected value of the measured property $M(Y)$ between the two conditions. That difference is not, however, the quantity of interest. A prompt that improves the answer to one question and degrades the answer to the next has established no effect at all, and a difference observed on a single input cannot be separated from the particular difficulty, phrasing or subject matter of that input. The research question asks about the behaviour of the system over the queries it will actually meet, so the effect has to be defined as an average over the distribution from which queries are drawn. Writing P_Q for that distribution over the query space $\mathcal{Q}$, the causal effect of prompt condition S_j relative to the baseline S_0 is

$$\tau_j = \mathbb{E}_{q \sim P_Q} \left[\mathbb{E}_{Y \sim p_{\text{RAG}}(\cdot \mid q, D, S_j)} [M(Y)] - \mathbb{E}_{Y \sim p_{\text{RAG}}(\cdot \mid q, D, S_0)} [M(Y)] \right], \quad (4.12)$$

which connects the probability model of Section 4.4 directly to the quantity the experiment estimates. Equation (4.12) is what Chapter 5 measures.

Equation (4.12) is an average treatment effect in the standard sense: the query plays the role of the unit, the prompt condition the role of the treatment, and τ_j the mean over units of the difference between the two treatments. Two features of how it is written matter. The difference is taken inside the outer expectation rather than between two separately averaged quantities. That is what makes each query its own control, and it matches the paired design of Section 4.6. By

linearity of expectation the two forms agree, but only the form given here records the pairing. The expectations are also nested rather than pooled: the inner expectation averages over the randomness of retrieval and decoding at a fixed query, while the outer averages over the queries themselves, so a large effect confined to a handful of inputs is not mistaken for a general one.

The distribution P_Q is not known and cannot be written down. Chapter 5 replaces it with the empirical distribution of a finite query set. That is the point at which the choice of data becomes a claim about generality rather than a matter of convenience, and it is why Section 4.8 draws the queries from established question-answer datasets rather than composing them for the experiment.

The choice of M is left open here. Section 4.9 describes what that property has to capture, and Section 5.7 fixes it.

4.8 Grounding the Knowledge Base in Established Datasets

The knowledge base has so far been written as a set $D = \{d_1, \dots, d_N\}$. Filling D is a methodological decision rather than a further step of formalisation. It is settled here because the effect defined in Equation (4.12) is only as informative as the corpus and the queries it is estimated over.

The route taken here is to instantiate D and the query set from established question-answer datasets. Three properties recommend it. The questions are naturally occurring rather than authored for the experiment. The corpora are public, versioned and split in a documented way, so a third party can repeat the experiment. And because published results for a wide range of retrieval and generation systems already exist on these datasets, the effect estimated in Chapter 5 can be read against a background of prior work instead of standing alone.

The collection is built from queries sampled from Bing search logs, with candidate passages retrieved for each query and free-form answers written by human judges (Nguyen et al., 2016). MS MARCO provides the queries q , the documents that populate D , and, through its annotations, both the reference answer a_q and the fact set against which an output can be judged.

What it does not provide is a definition of a good answer. An annotation records which answer is correct; it says nothing about whether an answer is grounded in the evidence actually retrieved, whether it addresses the question directly rather than evading it, or whether it obeys the requirements the system prompt states. Those properties are the object of this thesis, and defining them is the subject of the next section.

4.9 What Counts as a Good Answer

Quality in a retrieval-augmented system is not one property but several, and they can fail independently of one another. The first is groundedness. An answer should rest on the documents the retriever actually supplied. The second is relevance. An answer should address the question that was asked and address it directly, so that hedging, padding and drift towards a neighbouring question all count as failures. The third is correctness. An answer should agree with what is in fact the case, which is a different demand from the first and is treated separately here for a reason given below. The fourth is obedience. Whatever the system prompt asks for should be visible in the output itself, whether in length, in form, or in the handling of missing evidence, since a prompt that is not followed cannot be said to have any effect on quality at all.

All four are properties of the text the generator produces. Each of the four is estimated by a measure taken from published work. Groundedness by faithfulness, which asks whether every claim can be traced to a retrieved passage; relevance by answer relevance, that asks whether the response

addresses the question put to it; correctness by agreement with the reference answer recorded in the data; and obedience by whether the output satisfies every instruction the prompt states.

The formal definitions this evaluation rests on are introduced in Chapter 5. That chapter states the set of acceptable answers precisely and fixes faithfulness, answer relevance, correctness and instruction adherence as the four measures that estimate the four properties. Each of them is used in turn as the measured property M of Equation (4.12), so the effect of a prompt condition on each is estimated separately. Chapter 5 also specifies the corpus, the query set, the experimental conditions and the procedure by which those effects are estimated.

5 Experimental Setup

Chapter 4 built a model of the RAG system and defined the effect of a prompt condition as the average treatment effect τ_j of Equation (4.12). This chapter turns that model into a concrete experiment. It states the research question and the hypothesis, describes the system that is tested and which of its parts are held fixed, defines the prompt conditions, explains how the data are collected, and fixes how the outputs are scored and analysed. All experiments are implemented in Python. The implementation follows the structure of this chapter, and every design decision stated here is checked by the code before any data are collected.

5.1 Research Question and Hypothesis

Chapter 1 asked to what extent systematic changes in the system prompt influence the behaviour and outcomes of a RAG system when the user input, the knowledge base, the retrieval mechanism and the language model are held fixed. The experiment puts that question in the form it can be tested in: *does a RAG system behave differently when the structure of its system prompt is changed while the meaning of the prompt stays the same?*

Structure means how the prompt is written: which words are used, how the sentences are built, and how the text is laid out on the page. Meaning means the set of requirements Σ that the prompt states, in the sense of Equation (4.9). Every condition of the experiment keeps Σ fixed and changes only the wording φ . If the system responded only to what a prompt requires and not to how it is written, all wordings of one prompt would produce the same distribution of answers. Whether they do is the empirical question. Stated as hypotheses:

H_0 . System prompts that state the same requirements in different wordings produce the same distribution of answer quality. For every baseline b and every wording j , $\tau_j = 0$.

H_1 . At least one rewording changes answer quality: $\tau_j \neq 0$ for some j .

The comparison is always between a reworded prompt $S_{b,j}$ and the canonical prompt $S_{b,0}$ of the same baseline, on the same queries and with the same retrieved evidence. What is measured is the change in answer quality, where quality is the four properties of Section 4.9: grounded, relevant, correct and obedient.

One secondary question is asked of the same data. Does the answer to the main question survive when the quality of the evidence changes? The evidence is therefore supplied at two levels, as a controlled factor rather than as a further treatment (Section 5.4).

5.2 Overview of the Experiment

The system under test is the single-shot RAG architecture of Chapter 2. It is used for the reason given in Section 2.5: in this architecture the system prompt reaches the answer along one path only, through the generator, so its effect can be identified. A query is embedded, the K most similar

passages are retrieved from a vector index, and a language model produces the answer from the system prompt, the passages and the query. Table 5.1 lists the components of that system and the value each one is fixed at.

Table 5.1: The components of the system under test and their fixed values. None of them changes between prompt conditions.

Component	Fixed value
Knowledge base D	MS MARCO v2.1 validation passages, $N = 122,678$
Chunking χ	one passage per chunk, as published
Embedding model	all-mpnet-base-v2 , mean pooling, L2-normalised
Vector index	FAISS flat inner-product index, exact search
Retrieval depth K	10
Generator θ	gpt-4o-mini via the chat completions API
Decoding	temperature 1.0 (the API default), at most 256 new tokens
Judge	gpt-4.1-mini , temperature 0, fixed prompts
Query set	100 MS MARCO queries, seed 20260904
Repetitions n	3 per cell

The experiment has three stages. In the first stage, retrieval is run once for every query and its result is written to disk. In the second stage, every prompt condition is applied to every query, reading the stored retrieval result, and the answers are written to disk. In the third stage, the answers are scored and the prompt effect is estimated. Section 5.6 describes the first two stages in detail and Section 5.7 the third.

5.3 System-Prompt Conditions

The system prompt is the only thing that changes between conditions. A condition is a pair of a baseline and a wording. This section first introduces the three baselines and then the four wordings each baseline is written in.

5.3.1 Three Published Baselines

The prompts are not written for this experiment. Each baseline is a system prompt published with a retrieval-augmented system, and its canonical form is reproduced here exactly as it appears in its source. The three sources state different numbers of requirements, and that is why three are used rather than one. A prompt with one requirement gives a change of wording little to act on. A prompt with ten requirements is long, and a wording effect found on it alone could be an effect of length. Running the same four wordings on all three separates the effect of wording from the effect of how much the prompt asks for: if rewording matters in the same way regardless of how much is asked, the effect should appear in all three, and if it does not, the difference between them is itself informative.

Baseline *A* is the response-generation prompt of Friel et al. (2024). It states one requirement: answer from the supplied context.

```
Use the following pieces of context to answer the question.
{documents}
Question: {question}
```

Listing 5.1: Baseline *A*: the response-generation prompt of Friel et al. (2024).

Baseline *B* is the VANILLA instruction of Gao et al. (2023b). It states the most. Beyond grounding, it constrains accuracy, engagement, concision and register, it fixes the syntax and the density of citations, and it warns that some of the supplied results may be irrelevant.

```
Write an accurate, engaging, and concise answer for the given
question using only the provided search results (some of which
might be irrelevant) and cite them properly. Use an unbiased and
journalistic tone. Always cite for any factual claim. When citing
several search results, use [1] [2] [3]. Cite at least one document
and at most three documents in each sentence. If multiple documents
support the sentence, only cite a minimum sufficient subset of the
documents.
```

Listing 5.2: Baseline *B*: the VANILLA instruction of Gao et al. (2023b), reproduced from their Table 23.

Baseline *C* is the instruction of Mao et al. (2024). Its authors describe it as three parts: the first directs the model to the retrieved passages, the second asks it to read the question carefully, and the third asks for an answer together with a justification. They report that this form outperforms both a bare instruction and a chain-of-thought variant on the same passages.

```
Refer to the passage below and answer the following question.
Make sure you fully understand the meaning of the question and
passages. Then give the answer and explain why you choose this
answer.
Passages: {passages}
Question: {question}
```

Listing 5.3: Baseline *C*: the comprehensive instruction of Mao et al. (2024).

The published prompts contain placeholders for the documents and the question. In this experiment the instruction text alone forms the system prompt *S* and is sent in the system role of the chat interface. The passages and the question are sent in the user role, in the fixed template of Listing 5.4. The template is the same in every condition, so the only text that differs between conditions is *S*. Passages are numbered from one, so that a citation instruction has something to refer to and so that the position of a passage can be recovered from the answer.

```
Passages:
[1] <first passage>
[2] <second passage>
...

Question: <the query>
Answer:
```

Listing 5.4: The user turn. It is identical in every condition; only the system turn *S* changes. With no passages, the line after **Passages:** reads (none).

5.3.2 Four Wordings of Each Baseline

Each baseline is written in four wordings. The first is the canonical form above. The other three each change one aspect of the wording and leave everything else as it is. The axes follow the

paraphrase types of Wahle et al. (2024), Zhan et al. (2024) and Kostić et al. (2026), as explained in Section 4.5. Table 5.2 states what each wording changes and the rule under which it was written.

Table 5.2: The four wordings. Every wording of a baseline states the same requirement set Σ_b .

j	Wording	What changes, and the rule kept
0	canonical	Nothing. The published form.
1	lexical	Words are replaced by synonyms. Clause order and voice are kept.
2	syntactic	Clauses are reordered and active voice becomes passive. The vocabulary is kept as far as English allows. Obligation is carried uniformly by <i>is to be</i> , so that the change of voice does not change how strongly something is required.
3	format	The same wording is laid out as a list with headings instead of as prose. List items are bare noun phrases, so that no obligation is added or removed.

The rules in the last column matter. A rewording that changed how strongly an instruction is stated, for example by turning *always cite* into *please try to cite*, would change what the prompt requires. Such a change is a change of Σ , and a difference in behaviour caused by it could not be read as an effect of structure. No condition of this experiment makes a change of that kind.

The four wordings of baseline *A* are given in Listing 4.1 in Chapter 4. Listings 5.5 and 5.6 give the three rewordings of baselines *B* and *C*; their canonical forms are Listings 5.2 and 5.3 above.

```

B1 Compose a precise, compelling, and succinct reply for the posed
    query drawing only on the supplied search results (a number of
    which may be immaterial) and reference them correctly. Adopt an
    impartial and journalistic register. Invariably reference every
    factual assertion. When referencing several search results,
    employ [1][2][3]. Reference at minimum one document and at
    maximum three documents per sentence. If several documents
    corroborate the sentence, only reference a minimal adequate
    subset.

B2 For the given question an accurate, engaging, and concise answer
    is to be written from the provided search results alone, some
    of which might be irrelevant, and cited properly. An unbiased,
    journalistic tone is to be used. A citation is always given for
    any factual claim. Where several results are cited, [1][2][3]
    is used. In each sentence at least one and at most three
    documents are cited. Where multiple documents support a
    sentence, only a minimum sufficient subset is cited.

B3 ANSWER
    - accurate, engaging, and concise, for the given question
    - source: only the provided search results
    - note: some of the provided results might be irrelevant

CITATIONS
    - cite the search results properly
    - always cite for any factual claim

```

- when citing several search results: [1] [2] [3]
- in each sentence: at least one document, at most three documents
- if multiple documents support the sentence: only a minimum sufficient subset

TONE

- unbiased and journalistic

Listing 5.5: The lexical ($B1$), syntactic ($B2$) and format ($B3$) wordings of baseline B .

C1 Consult the excerpt below and respond to the following query.
Ensure you completely grasp the sense of the query and excerpts.
Then supply the response and justify why you select this response.

C2 The following question is to be answered from the passage below.
The meaning of the question and passages is first understood.
The answer is then given, with an explanation of why it is chosen.

C3 SOURCE: the passage below, and nothing else
READ: fully understand the meaning of the question and passages
ANSWER: give the answer to the following question
EXPLAIN: say why you choose this answer

Listing 5.6: The lexical ($C1$), syntactic ($C2$) and format ($C3$) wordings of baseline C .

Prompt length is held nearly constant within a baseline, so that a difference between two wordings cannot be an effect of how much text the prompt contains. Table 5.3 gives the word counts. No wording departs from its canonical form by more than one tenth, and the code refuses to run if an edit to a prompt breaks that bound.

Table 5.3: Word count of every condition, with the deviation from the canonical form of its baseline.

Baseline	canonical	lexical	syntactic	format
A	10	10 ($\pm 0\%$)	11 (+10%)	11 (+10%)
B	74	71 (-4%)	80 (+8%)	77 (+4%)
C	33	32 (-3%)	34 (+3%)	32 (-3%)

Twelve conditions result: three baselines in four wordings each. They are written $S_{b,j}$ with $b \in \{A, B, C\}$ and $j \in \{0, 1, 2, 3\}$. Every comparison is made within a baseline. Comparing $S_{B,1}$ with $S_{A,0}$ would mix a change of wording with a change of what is asked for, so each rewording is compared only with the canonical form of its own baseline.

5.4 Variables and Controlled Factors

The experiment has one independent variable, one dependent variable, one controlled factor that is varied for robustness, and a set of components that are held constant.

Independent variable. The wording j of the system prompt, with four levels, nested within the baseline b . Changing j changes the structure of the prompt and nothing else. The requirement set Σ_b is the same at every level of j .

Dependent variable. The quality of the answer, measured by the four measures of Section 5.7: faithfulness, answer relevance, correctness and instruction adherence. Each is used in turn as the measured property M of Equation (4.12), so the effect of a rewording is estimated four times, once per measure, and the four are never combined into a single score. Pooling them would hide direction: a rewording that improved grounding and damaged relevance by the same amount would average to no effect at all.

Controlled factor. One factor is varied so that the main question can be asked under more than one condition of the evidence. It is not a treatment. The prompt effect is estimated separately at each of its levels, and the levels are not compared with one another as a result of their own.

That factor is the **evidence level**. The context C handed to the generator is set directly, as the intervention $\text{do}(C = c_{ik})$ of Chapter 4, at two levels:

c^{gold} . Only the passages the annotation marks as supporting the answer. The evidence needed for the answer is certainly present, so a failure here belongs to the generator rather than to retrieval. Every query is usable at this level, which makes it the more sensitive of the two.

c^{ret} . The K passages the fixed retriever actually returns, gold and distractors alike. This is the system as it would be deployed.

Both levels are produced by setting the context, never by changing the corpus and retrieving again, so the index and the retriever are identical for both.

The retrieval depth is not a second factor. It is held constant at $K = 10$. That depth admits more of the annotated evidence than a shallower one, as Table 5.4 shows, so more queries are usable at the retrieved level and the comparison there rests on a larger sample; and ten passages give the prompt a longer and noisier context to govern than five would. A second depth was considered as a further robustness factor, but it would have doubled the size of the run to answer a question secondary to the one being asked, and it is not part of this design.

Held constant. Everything in Table 5.1. The generator is one model behind one API, with a fixed temperature and a fixed output limit. Queries and passages are encoded by one embedding model, applied to both sides in the manner of the dense retriever of Karpukhin et al. (2020). The index, built with the library of Johnson et al. (2021), is constructed once and never rebuilt. Each passage is one chunk. Because the embedding model, the index, the chunking and the depth are all fixed, the set $\mathcal{Z}_K(q, D)$ of Equation (4.2) depends only on the query and the depth and never on the system prompt. The context is held identical in a stronger sense than that. Not only the set of passages but also their order are the same in every condition: they are determined by the query, the evidence level, the depth and the repetition, and by nothing else. Two conditions compared on the same query therefore receive a character-for-character identical user turn, and the only text that differs between them is the system prompt. Any change in the answer originates at the generator, which is the localisation the intervention of Section 4.6 assumes.

Covariates. Two properties of each query are recorded and can be conditioned on in the analysis: the category (direct or multi-hop) and MS MARCO’s intent type. Both are defined in Section 5.5.

The design. An observation is one answer, written

$$y_{ijk r} \sim p_{\text{PRAG}}(y \mid q_i, \text{do}(C = c_{ik}), \text{do}(S = S_j)), \quad r = 1, \dots, n, \quad (5.1)$$

for query i , prompt condition j , evidence level k and repetition r . The twelve prompt conditions and the two evidence levels are fully crossed, which gives 24 cells. Every cell is run for all 100 queries and repeated $n = 3$ times, which is 7,200 generations before the retrieval gate of Section 5.6 removes any.

Within a cell the only remaining source of variation is the decoding randomness ε of Equation (4.10). Each repetition is an independent draw from the same distribution, so averaging n of them leaves the quantity being estimated unchanged and reduces the standard error of the cell mean by a factor of $\sqrt{n}$. Three repetitions is fewer than the five Zhuo et al. (2024) use for the same purpose, and the cost of that choice is stated in Section 5.8: a cell mean taken over three draws is noisier than one taken over five, so a real effect is harder to separate from the noise of sampling.

5.5 Knowledge Base and Query Set

Following Section 4.8, both the knowledge base and the query set come from MS MARCO (Nguyen et al., 2016), read from the Hugging Face export of version 2.1. The validation split is used. It holds 101,093 queries, of which 12,467 carry a non-empty `wellFormedAnswers` field. Only those rows are used, because the well-formed answer is the reference string against which correctness is measured, and a query without one cannot be scored.

The knowledge base. The corpus is defined by a rule, not by a sample: it holds every passage attached to any of the 12,467 rows. MS MARCO ships about ten candidate passages per query, retrieved by Bing and then judged by annotators, so the corpus contains the gold evidence for every query together with the near-misses that were retrieved alongside it. Identical passage texts are indexed once; 124,316 passages become $N = 122,678$ after deduplication. There is no distractor pool to size, and the gold passages of a query compete against the whole corpus. Each passage is one chunk and is used at the length at which it was published. The median passage has about 50 words and three sentences; eight in ten passages have between two and six sentences.

The query set. $N_Q = 100$ queries are sampled from the same 12,467 rows. They fall into two categories, defined by the annotation rather than by any judgement made here. A row is *direct* when exactly one of its passages is marked `is_selected`, and *multi-hop* when two or more are. Fifty queries are drawn from each category.

$\mathcal{Q}_{\text{direct}}$. The answer is carried by a single passage. One passage suffices, so a failure here is a failure to use evidence that was present. This is the noise-robustness testbed of Chen et al. (2024).

$\mathcal{Q}_{\text{multi-hop}}$. The human-written answer draws on two or more passages. The annotation identifies which, so the need to combine evidence is recorded rather than asserted. This is the information-integration testbed of Chen et al. (2024).

Within each category the draw is balanced across MS MARCO’s five intent types, DESCRIPTION, ENTITY, LOCATION, NUMERIC and PERSON, ten queries per type. The natural distribution is very uneven: on the well-formed rows DESCRIPTION makes up 44% and PERSON 7%. An unbalanced draw would leave the rarer types with too few queries to say anything about, and would let an aggregate figure be carried by one type. Balancing makes the type usable as a covariate. The draw uses the seed 20260904 and is reproducible from the same split.

The multi-hop half is drawn from a much smaller pool than the direct half. Of the 12,467 well-formed rows, 12,062 are direct and only 386 are multi-hop; 19 have no passage marked at all and are

dropped. The thinnest cell, multi-hop PERSON, holds 27 candidates for the ten drawn. The multi-hop sample is therefore a substantial fraction of what exists, which bounds how much a different seed could change it.

For every query the reference answer a_q is its first well-formed answer, and its gold passages are the ones marked `is_selected`. Because the corpus is deduplicated, a gold passage whose text also appears under an earlier row is indexed under that earlier row’s identifier; the gold identifiers are rewritten accordingly, so that the retrieval check of the next section looks for an identifier that exists.

5.6 Experimental Procedure

The experiment runs in two data-collection stages. Retrieval is finished and written to disk before any prompt condition exists. Generation reads that stored output and never calls the retriever. This separation is what enforces the assumption of Section 4.6 that the system prompt has no effect on which documents are retrieved: there is no execution path along which a prompt could reach the retriever.

5.6.1 Stage 1: Retrieval and the Retrieval Gate

Queries and passages are encoded with `all-mpnet-base-v2`, using mean pooling over the last layer and L2 normalisation, so that the inner product of two vectors is their cosine similarity. Inputs are truncated at 384 tokens. The index is a flat inner-product index: every query is compared against every passage, with no approximation. At 122,678 vectors of 768 dimensions the index occupies 0.38 GB and fits in memory, and exactness matters more than speed here, because an approximate index would make retrieval depend on how the index was built rather than on the query alone.

Retrieval is run once, at $K = 10$, for all 100 queries. The identifiers, scores and ranks of the returned passages are written to disk.

The retrieval stage is also where the evidence is checked. A query passes the *retrieval gate* at a depth when every passage its annotation marks as supporting appears among the K returned at that depth. For a direct query that is one passage; for a multi-hop query it is every passage the answer draws on, because an answer that needs two passages cannot be produced from one. Table 5.4 reports the result. It is a property of the fixed retriever and the corpus, not of any prompt condition, and it is reported before the results because it sets the denominator of the retrieved level.

Table 5.4: The retrieval gate at the depth used, $K = 10$. A query passes when all of its gold passages are among the ten returned. Gold recall counts the gold passages that arrived, over all 152 gold passages of the query set. The figures for $K = 5$ are given for comparison; that depth is not run.

K	Passed	direct	multi-hop	Gold recall
10 (used)	88 / 100	45 / 50	43 / 50	139 / 152 (91.4%)
5	74 / 100	41 / 50	33 / 50	123 / 152 (80.9%)

Queries that fail the gate at a depth are excluded from the retrieved level at that depth. The measures of Section 5.7 are not defined on evidence that was never supplied: when the context lacks the passage carrying the answer, a correct answer can only have come from the generator’s memory, and an incorrect answer cannot be told apart from a faithful reading of the other passages. Scoring such a cell would measure the retriever and report the result as an effect of the prompt.

The exclusion is applied to all twelve conditions at once, because retrieval is identical across them, so the pairing survives: every comparison is still made within a query, and no condition is evaluated on queries another condition was not. The gold level sets the context directly, cannot fail retrieval, and runs over the full query set.

5.6.2 Stage 2: Generation

For every query, every cell of the design is run $n = 3$ times. The cells are visited in a random order per query, so that any drift in the API over the collection period cannot align with a condition. Every generation is an independent request; nothing is reused between requests, and no random seed is sent, so the repetitions are independent draws. The context for a cell is built as follows.

- At c^{gold} , the passages are the query’s gold passages.
- At c^{ret} , the passages are the top K from the stored retrieval output at that depth.

The order of the passages within the context is shuffled, and the positions of the gold passages are recorded, because answer accuracy depends on where the relevant passage sits (Liu et al., 2024). The shuffle is drawn from a generator seeded on the query, the evidence level, the depth and the repetition, and deliberately not on the prompt condition. That is what makes the context identical across conditions. Because the seed still varies with the repetition, the order differs between the three repetitions of a cell, so the mean of a cell is an average over three arrangements rather than a figure tied to one; and because it does not vary with the condition, that same set of three arrangements is used in every condition. The system prompt $S_{b,j}$ is sent in the system role and the passages and question in the user role, in the template of Listing 5.4. The generator is `gpt-4o-mini` at temperature 1.0 with at most 256 new tokens.

For each generation the following are written to disk: the query and its category and type; the condition, evidence level, depth and repetition; the identifiers and order of the passages in the context and the positions of the gold passages; the exact system prompt; the model identifier, the decoding settings and the backend fingerprint the API returned; the answer and its token count; and a timestamp. Nothing needed to reconstruct a generation is left implicit.

5.6.3 The Pipeline

In summary, the experiment proceeds in the following steps.

1. Download the MS MARCO v2.1 validation split and keep the 12,467 rows with a well-formed answer.
2. Embed every passage attached to those rows and build the index: 122,678 passages.
3. Sample the query set: 50 direct and 50 multi-hop queries, ten per intent type, seed 20260904.
4. Retrieve the top 10 passages for every query and apply the retrieval gate. The top 5 are retrieved as well, so that the gate can be reported at that depth for comparison in Table 5.4, but no generation is run at that depth.
5. For every query, every one of the 24 cells and every one of the 3 repetitions, build the context, send $S_{b,j}$ and the user turn to the generator, and record the answer.
6. Score every answer with the four measures of Section 5.7.

7. Estimate the effect of each rewording in every cell, once per measure, with a confidence interval, and correct for multiple comparisons.

5.7 Evaluation

Many different texts can express the same answer, so the standard defines a set of acceptable answers $\mathcal{Y}^*(q, C)$ rather than one ideal string. An answer belongs to that set when it satisfies four requirements at once:

Grounded. It asserts only what the supplied passages state, and introduces nothing from outside them.

Relevant. It addresses the question that was asked, and where the evidence will not support an answer it says so rather than producing something off-target.

Correct. It agrees with the reference answer of the dataset annotation.

Obedient. It satisfies every requirement the system prompt states.

An answer that breaks none of the four is a good answer. One that breaks several is worse than one that breaks one. Each requirement is estimated by one measure. All four measures are taken from published RAG evaluation work rather than defined here. The first three are the measures that the survey of Yu et al. (2024) names as the evaluation targets of the generation component: faithfulness relates the response to the retrieved documents, answer relevance relates it to the query, and correctness relates it to the reference answer. Retrieval measures are excluded by construction, because the system prompt cannot change what is retrieved.

5.7.1 The Four Measures

Faithfulness. Whether every claim in the answer can be traced to a supplied passage. It is scored reference-free, after Es et al. (2023), in two steps. First the judge (Section 5.7.2) breaks the answer into atomic factual claims. Then, for each claim, the judge decides whether the passages state it or directly entail it. The breach count $n_1(y)$ is the number of unsupported claims; the reported rate is the supported fraction, because a raw count grows with answer length and prompts change how much a model writes. This is the measure the experiment turns on. Every one of the twelve conditions states a grounding constraint, and faithfulness shows whether that constraint is followed in practice. Because the passages are identical across conditions, a change in faithfulness cannot come from a change in what the generator was given.

Answer relevance. Whether the answer addresses the question actually asked, and directly. A prompt can degrade an answer without making it false, by causing hedging, padding, or drift towards a neighbouring question. Relevance is scored reference-free, after Es et al. (2023): the judge writes three questions that the answer would be a good answer to, and those are compared with the question actually asked by their mean cosine similarity in the space of the retrieval encoder, so that no second embedding space is introduced. A mean below 0.5 counts as a breach. Refusals are treated separately. Both evidence levels of this design supply passages, so an answer that declines to answer has declined in the presence of evidence and counts as a breach. A refusal is detected by a fixed list of patterns such as *cannot be answered* and *not enough information*, because none of the three baselines mandates a fixed abstention string.

Correctness. Whether the answer agrees with the reference answer a_q , in the sense of Chen et al. (2024): the answer is correct when, after lowercasing and removing punctuation and articles, it contains the reference. Exact match is not used, because MS MARCO’s well-formed answers are full sentences and an answer that says the same thing in other words would be counted wrong. Containment weakens that demand without removing it. The reference must still appear inside the answer word for word, so a paraphrase is still read as an error. Token-level F1 is therefore reported beside containment as a secondary measure, and exact match is recorded for inspection. F1 compares the two strings word by word and survives a rephrasing that containment does not. Section 5.8 states how far the two come apart on this data and why the first of them keeps its place.

Correctness and faithfulness are separate demands. An answer can repeat a supplied passage accurately and still be wrong about the world, and it can be right about the world while resting on nothing the retriever returned. Measuring both keeps the two apart.

Instruction adherence. Whether the answer does what the prompt told it to. It is scored as prompt-level strict accuracy in the sense of Zhou et al. (2023): an answer is adherent only if it satisfies every checked requirement at once, so the score is binary. Only requirements that a rule can decide are checked, which is the principle of verifiable instructions, and a condition is held only to what its own baseline states. For baseline *B* the rule parses the citation markers and checks that every sentence carries between one and three of them, which is what the prompt asks for. For baseline *C* the rule checks that the answer contains a justification, marked by words such as *because*, *since* or *therefore*. Baseline *A* states only grounding, which faithfulness already measures, so its adherence is undefined by construction rather than a free pass. Grounding is never checked inside adherence for any baseline, because counting it there as well would count one failure twice. Requirements that no rule can decide, such as an unbiased tone, are recorded as not checked. The requirement set is the same for all four wordings of a baseline, so adherence is directly comparable across them.

5.7.2 The Judge

Faithfulness and answer relevance have no reference string, so they are scored by a second language model, as Es et al. (2023) and Saad-Falcon et al. (2024) do. The judge is `gpt-4.1-mini`, a different model from the generator, so that the generator never grades its own output, and it is run at temperature 0 so that a verdict depends on the answer and the passages and not on a random draw. Since this thesis assumes that language models are sensitive to their instructions, the judge cannot be assumed reliable. Three precautions are taken, following Zheng et al. (2023), who document position and verbosity biases in this setting. The judge’s own prompts are fixed constants and are never varied between conditions. The judge is shown only the answer and the passages, never the condition label, the query identifier or the evidence level; this matters because the format condition is recognisable from its surface alone, and a judge error that varied with condition would bias the estimated effect. And the answers are judged in random order, so that position in the judging sequence cannot correlate with condition.

A fourth precaution is specified but was not carried out. The design provides for an agreement study against human labelling: about 100 scored answers are drawn evenly across the twelve conditions, the judge’s own verdict is withheld from the labeller, and agreement is reported as Cohen’s kappa for each condition separately rather than pooled, because a pooled figure can look reassuring while hiding an error that varies with condition. The sample is drawn by the implementation, but the labelling itself is manual and was not completed within the scope of this thesis. The two

judged measures therefore carry no reliability estimate, which Section 5.8 states as a limitation and Section 7.3 names as the first piece of work to follow.

5.7.3 Estimating the Effect of a Rewording

The quantity reported is the average treatment effect of Equation (4.12). Chapter 4 left the measured property M open. It is fixed here as each of the four measures in turn, so the effect of a rewording is estimated four times, once for faithfulness, once for answer relevance, once for correctness and once for instruction adherence. Every measure is oriented so that a higher value is better, and a positive effect therefore means the rewording improved that measure.

The four are never added together. A single combined score would hide the direction of a change: a rewording that improved grounding and damaged relevance by the same amount would produce no combined effect at all, and the thesis would report that nothing happened when in fact two things did.

For one measure M , one evidence level and one baseline b , the estimate of the effect of wording j is

$$\hat{\tau}_j = \frac{1}{|I|} \sum_{i \in I} (\bar{M}_{ij} - \bar{M}_{i0}), \quad \bar{M}_{ij} = \frac{1}{n} \sum_{r=1}^n M(y_{ijr}), \quad (5.2)$$

where I is the set of queries in that cell and $\bar{M}_{ij}$ is the mean of the measure over the n repetitions of query i under condition $S_{b,j}$. Equation (5.2) is the sample version of Equation (4.12): the inner average takes the expectation over decoding, and the outer one takes it over queries.

Two properties of that formula matter. The difference is taken inside the sum, so every query is compared with itself and the variation between queries drops out. That is the pairing the design was built for, and it is possible only because every query is run in every condition on an identical context. And the repetitions are averaged before the difference is taken, so one query contributes one number. The repetitions measure decoding noise rather than variation between queries, and treating them as independent observations would make the sample look three times larger than it is.

A measure that does not apply to a generation is skipped rather than counted as satisfied. This affects instruction adherence under baseline A , whose only requirement is grounding: $\hat{\tau}$ for adherence is therefore reported for baselines B and C only.

Correctness is estimated twice, once by containment and once by token F1. Containment is the measure Section 5.7 fixes in advance and it is the one that decides the hypothesis. F1 is secondary and is reported beside it. The two are corrected separately, each within its own family of nine rewordings, and the decision of Section 5.7.4 reads the four measures named above and nothing else. Choosing between them after seeing the results would make the hypothesis depend on which measure gave the clearer answer, so both are reported and the measure fixed in advance keeps its place.

5.7.4 Deciding the Hypothesis

$\hat{\tau}_j$ is an average over a finite sample, so it is almost never exactly zero even when the true effect is. A point estimate on its own therefore cannot decide between the hypotheses of Section 5.1. Each estimate is reported with its standard error and a 95% confidence interval, computed from the same paired differences that produced it. An interval that excludes zero is evidence that the rewording changed that measure; one that contains zero is not.

The hypothesis H_1 claims that *at least one* rewording changes answer quality, and it ranges over nine rewordings for each measure. Reading nine intervals and concluding H_1 because one of them excluded zero would report chance as a finding: at the 5% level roughly one interval in twenty excludes zero when nothing is happening. The intervals are therefore corrected by the method of Holm (1979) across the nine rewordings within each measure, which is the family the claim ranges over. Both the uncorrected and the corrected verdict are reported, so a reader can see what the correction changed.

H_0 is rejected if, after correction, at least one rewording moves at least one measure. If no interval survives correction, H_0 is not rejected. That is a weaker statement than showing the effect to be zero, and Section 5.8 states how large an effect this design could have detected.

Alongside the estimates, the mean of each measure in each cell is reported, so that an effect can be read against the level it moves from. A change of two points means something different at a faithfulness of 0.9 than at one of 0.4.

5.8 Limitations

The evaluation rests on a single collection. Every result is conditional on MS MARCO: on its queries, its passages, its relevance judgements and its written answers. Section 4.8 argues that the topical spread within the collection is wide, but spread within one corpus is not replication across several. A prompt effect found here bounds what may be expected elsewhere; it does not establish it.

The query set is balanced across intent types. That makes the type usable as a covariate, but it detaches the query set from the natural distribution of queries: PERSON and LOCATION queries are over-represented relative to MS MARCO and DESCRIPTION queries under-represented. The prompt effect τ_j is unaffected, because it is an average of within-query differences and each query is its own control. Any marginal figure over the query set is a balanced average over the design, not an estimate of what the system would do on the queries a deployment receives.

MS MARCO records what people type into a web search engine. Its queries are short, mostly factoid, and drawn from the domains web search covers. Several of the settings that motivate retrieval in Section 2.4, such as clinical decision support and legal question answering, are absent from it, and their documents differ from web passages in length, structure and register. Whether an instruction that governs behaviour here governs it there is left to future work.

The design can detect a medium effect but not a small one. With the 88 queries that pass the retrieval gate at $K = 10$, a paired comparison at the 5% level has an 80% chance of detecting a standardised effect of about 0.30, and about 0.39 once the correction across the nine rewordings is applied; at the gold level, where all 100 queries are usable, the corresponding figures are 0.28 and 0.37. Those thresholds are stated in units of the observed paired differences. Two choices make those differences noisier than they need to be: three repetitions per cell rather than five, and a decoding temperature of 1.0 rather than a low one. Both raise the variation within a cell, so a given true effect is harder to detect. If a rewording changes a measure by less than that, this experiment will not distinguish it from decoding noise, and a failure to reject H_0 has to be read as that limit rather than as evidence that prompt structure does not matter.

The containment form of correctness cannot see through a paraphrase, and on this data that limit binds hard. MS MARCO’s reference answers are full sentences, the generator rarely reproduces one word for word, and a correct answer written in other words scores zero. The measure therefore sits near the bottom of its range in every condition, at a mean of 0.053 against 0.012 for exact match, and

849 of the 6,768 generations score zero on it while their token F1 exceeds 0.6. A measure with that little room to move is insensitive to the size of change this experiment produces, and what it does respond to is phrasing, which is the very thing the rewordings alter. That is why the secondary F1 estimate of Section 5.7.3 is reported alongside. Where the two disagree, the disagreement describes the instrument rather than the system: a null on containment says this measure could not detect a change, not that no change occurred. Neither measure establishes truth. F1 rewards overlapping words, so a fluent wrong answer that borrows the vocabulary of the question can score above a short right one.

The generator and the judge are models behind an API. Their weights are not available, and the provider may update a model under the same name. The model identifier, the decoding settings and the backend fingerprint returned by the API are recorded with every generation, so any such change is visible in the data, but an exact repetition of the run depends on the provider keeping the model unchanged.

Two of the four measures are judged by a language model. The precautions of Section 5.7.2 limit the ways in which the judge can bias the comparison between conditions, but they do not remove the dependence on it, and the agreement study those precautions call for was not carried out, so faithfulness and answer relevance carry no reliability estimate. The two measures that carry the result of Chapter 6, correctness and instruction adherence, are decided by rule and do not depend on the judge, which limits how far this weakness reaches. Instruction adherence, in turn, checks only the requirements a rule can decide; requirements such as an unbiased tone are stated by the prompts but not scored.

6 Results and Discussion

Chapter 5 fixed the experiment: twelve system prompts, two evidence levels, a fixed retriever and generator, and four measures. This chapter reports what came out of it. It first describes the data that were collected and how the system behaved before any two conditions are compared, because an effect can only be read against the level it moves from. It then gives the effect of each rewording, measure by measure, and states the verdict on the hypothesis of Section 5.1. The last two sections look at what changed inside the answers themselves, and at what these results do not establish.

6.1 What Was Collected

The run produced 6,768 answers. At the gold level all 100 queries are usable, which gives $100 \times 12 \times 3 = 3,600$ answers. At the retrieved level 88 of the 100 queries pass the gate of Section 5.6 at $K = 10$, which gives $88 \times 12 \times 3 = 3,168$. Every answer was scored on the four measures of Section 5.7, and the two judged measures took 30,636 calls to the judge model.

Three checks were run on the collected data before any of it was analysed. Every one of the twelve conditions received a byte-identical user turn for the same query, repetition and evidence level, which is the property Section 5.4 requires and the one the comparison depends on. No answer appears twice. No answer is empty. Together these say that the only thing that differs between two conditions in the same cell is the system prompt.

6.2 How the System Behaved Before Any Comparison

Table 6.1 gives the mean of each measure in each cell. It is worth reading before the effects, because an effect of two points means something different at a faithfulness of 0.92 than at an adherence of 0.08.

Three things stand out. Faithfulness is high everywhere, between 0.912 and 0.976. Relevance is stable, between 0.693 and 0.746. Both sit in a narrow band, so neither has much room to move. Adherence is the opposite. It runs from 0.083 to 0.898, a spread of more than eighty points across conditions that ask for the same thing. Whatever the prompt is doing, this is where it shows.

Correctness measured by containment is near zero in every cell, from 0.015 to 0.120. Section 5.8 explains why. The measure asks whether the reference answer appears inside the generated answer word for word, and the generator almost never repeats a full MS MARCO sentence. The secondary F1 estimate in the fourth column is the same comparison made word by word, and it moves in a range four to five times wider.

One pattern in the table runs against expectation. Faithfulness is higher at the retrieved level than at the gold level, in every condition. The gold context contains only the passages annotated as necessary, so it is the easier setting for answering the question, but it is the harder setting for being fully supported: with fewer passages in front of it, more of what the model writes has no source in the context to point at. The retrieved context at $K = 10$ carries more text, and more text gives

Table 6.1: Mean of each measure per condition. The gold level rests on 100 queries and the retrieved level on the 88 that pass the gate at $K = 10$. Adherence is undefined for baseline A , whose only requirement is grounding.

	Faithfulness	Relevance	Correctness	Correctness F1	Adherence
<i>gold</i> , $ I = 100$					
A0	0.917	0.736	0.073	0.529	n/a
A1	0.922	0.730	0.070	0.509	n/a
A2	0.930	0.734	0.087	0.538	n/a
A3	0.930	0.741	0.087	0.528	n/a
B0	0.912	0.696	0.030	0.345	0.223
B1	0.942	0.705	0.023	0.387	0.447
B2	0.924	0.703	0.040	0.377	0.343
B3	0.931	0.703	0.020	0.383	0.297
C0	0.924	0.729	0.077	0.241	0.890
C1	0.919	0.738	0.077	0.252	0.447
C2	0.941	0.728	0.093	0.278	0.433
C3	0.939	0.731	0.120	0.296	0.817
<i>retrieved</i> , $K = 10$, $ I = 88$					
A0	0.968	0.737	0.023	0.423	n/a
A1	0.976	0.734	0.038	0.401	n/a
A2	0.974	0.732	0.042	0.425	n/a
A3	0.976	0.736	0.045	0.419	n/a
B0	0.962	0.693	0.015	0.219	0.083
B1	0.971	0.705	0.015	0.275	0.345
B2	0.957	0.696	0.023	0.251	0.193
B3	0.965	0.702	0.027	0.264	0.205
C0	0.963	0.734	0.042	0.176	0.898
C1	0.973	0.738	0.049	0.183	0.568
C2	0.973	0.746	0.061	0.203	0.473
C3	0.959	0.730	0.061	0.236	0.886

more claims something to rest on. This is an observation about the data, not something the design was built to test, and it is not part of the hypothesis.

6.3 The Effect of Rewording

Each rewording is compared with the canonical form of its own baseline, using Equation (5.2). Every comparison is paired within a query. The results are grouped by measure below, because that is the family the correction of Section 5.7.4 is applied within.

6.3.1 Instruction Adherence

This is where the effect lives. Table 6.2 gives all six estimates in both cells. It covers baselines *B* and *C* only. Baseline *A* states one requirement, that the answer come from the supplied context, and faithfulness already measures exactly that, so there is no second rule left for adherence to check. Section 6.6 returns to what that costs.

Table 6.2: Effect of each rewording on instruction adherence, against the canonical form of its own baseline. A star marks an interval that excludes zero after the correction across the nine rewordings in the measure.

Rewording	gold		retrieved, $K = 10$	
	$\hat{\tau}$	95% CI	$\hat{\tau}$	95% CI
B1 lexical	+0.223*	[+0.158, +0.289]	+0.261*	[+0.187, +0.336]
B2 syntactic	+0.120*	[+0.071, +0.169]	+0.110*	[+0.045, +0.175]
B3 format	+0.073*	[+0.026, +0.121]	+0.121*	[+0.058, +0.185]
C1 lexical	-0.443*	[-0.521, -0.366]	-0.330*	[-0.410, -0.249]
C2 syntactic	-0.457*	[-0.534, -0.379]	-0.424*	[-0.501, -0.348]
C3 format	-0.073*	[-0.142, -0.005]	-0.011	[-0.071, +0.049]

Eleven of the twelve estimates exclude zero after correction. Ten of them do so in both cells, which means the same rewording produced the same change on two different sets of evidence. Only C3 at the retrieved level fails to, and it is the smallest effect in the table.

The sizes are not small. Rewording baseline *C* with synonyms costs it 44 points of adherence at the gold level, and rewriting the same requirements in the syntactic form costs it 46. Baseline *B* moves the other way: every rewording of it scores higher than the published form, and the lexical rewording gains 22 points at the gold level and 26 at the retrieved level.

6.3.2 Faithfulness

One estimate of nine survives correction: B1 at the gold level, $\hat{\tau} = +0.031$ with a 95% interval of [+0.014, +0.047]. The same comparison at the retrieved level gives +0.009 with an interval that contains zero. A result that appears in one cell and not the other is weak evidence. Given the ceiling reported in Section 6.2, where faithfulness never falls below 0.91, it should not be read as more than that.

6.3.3 Answer Relevance

Nothing survives correction, in either cell. The largest estimate is +0.012. Relevance sits between 0.693 and 0.746 in every condition and the rewordings do not move it.

6.3.4 Correctness

Measured by containment, nothing survives correction, in either cell. All eighteen intervals contain zero and most of them are wide.

That null belongs to the measure. Table 6.3 gives the same estimates computed on the secondary token F1 measure of Section 5.7.3, on the same answers, with the same estimator and the same correction.

Table 6.3: Effect of each rewording on correctness measured by token F1, the secondary measure. Reported beside the containment estimates, not in place of them, and not part of the decision on H_0 .

Rewording	gold		retrieved, $K = 10$	
	$\hat{\tau}$	95% CI	$\hat{\tau}$	95% CI
A1 lexical	-0.019*	[-0.032, -0.007]	-0.022*	[-0.035, -0.009]
A2 syntactic	+0.010	[+0.000, +0.020]	+0.002	[-0.011, +0.014]
A3 format	-0.000	[-0.013, +0.012]	-0.004	[-0.013, +0.004]
B1 lexical	+0.041*	[+0.026, +0.056]	+0.056*	[+0.036, +0.077]
B2 syntactic	+0.032*	[+0.019, +0.045]	+0.032*	[+0.015, +0.049]
B3 format	+0.038*	[+0.025, +0.051]	+0.045*	[+0.027, +0.064]
C1 lexical	+0.011*	[+0.003, +0.018]	+0.006	[-0.000, +0.013]
C2 syntactic	+0.036*	[+0.028, +0.044]	+0.027*	[+0.020, +0.034]
C3 format	+0.055*	[+0.044, +0.065]	+0.060*	[+0.051, +0.068]

Thirteen of the eighteen estimates exclude zero, and twelve of those are six rewordings that hold in both cells. The two measures are asking about the same property of the same answers and they disagree completely. The disagreement is a statement about the instruments, not about the system. Containment could not detect a change here; that is different from there being none.

6.4 The Verdict on the Hypothesis

Sixty-six estimates were made across the four measures fixed in Section 5.7. Twelve exclude zero after the correction of Section 5.7.4: eleven on adherence and one on faithfulness.

H_0 states that prompts stating the same requirements in different wordings produce the same distribution of answer quality, so that $\tau_j = 0$ for every baseline and every wording. That is rejected. H_1 claims that at least one rewording changes answer quality, and eleven rewordings do, most of them in both cells.

The rejection does not rest on the single faithfulness result, nor on the secondary correctness measure, neither of which entered the decision. It rests on instruction adherence, where the effects are large, replicated across two evidence levels, and present in both directions.

That replication also answers the secondary question of Section 5.1, which asked whether the answer to the main question survives a change in the quality of the evidence. It does. Ten of the eleven adherence effects appear at both evidence levels, and their sizes are close: B1 gains 0.223 on gold and 0.261 on the retrieved context, C2 loses 0.457 and 0.424. Giving the generator the real output of the retriever, distractors included, rather than only the passages the annotation marks as supporting, does not remove the effect of how the prompt is worded.

6.5 Reading the Adherence Result

An effect of forty points needs an explanation of what changed in the text. This section gives one for each baseline. It describes what the answers look like; it does not add anything to the test above.

6.5.1 Baseline B: The Model Cites, But Not in Every Sentence

Baseline *B* asks for a citation in every sentence, and the rule of Section 5.7 follows it strictly: one sentence without a citation fails the whole answer. Under the canonical form only 15.8% of answers pass. That figure is easy to misread. The model does cite. A citation marker appears somewhere in 98.6% of the canonical answers. What it fails to do is cite in every sentence: across all of B_0 , 53.0% of sentences carry a marker.

The rewordings change that coverage. Under B1 it rises to 65.3%. Measured per sentence rather than per answer, the effect of B1 is +0.153 at the gold level and +0.150 at the retrieved level, both excluding zero. So the improvement in coverage is real.

It is not the whole story, because the strict rule is easier to pass with fewer sentences to get right, and B1 writes fewer. Canonical answers run to 3.1 sentences on average and B1 answers to 2.5. Part of the +0.223 gain in Table 6.2 is that shortening, and part is better coverage. The per-sentence estimate separates the two: the coverage gain survives on its own, at about two thirds the size.

6.5.2 Baseline C: Compliance Falls When the Instruction Stops Being an Instruction

Baseline *C* asks for three things, and the rule checks the one a rule can decide: the answer must explain its choice. Under the canonical form 89.4% of answers do. Under C1 that falls to 50.4% and under C2 to 45.2%.

The two prompts that lose compliance are the two that move furthest from a direct instruction. C0 says *give the answer and explain why you choose this answer*. C1 says *supply the response and justify why you select this response*, which asks for the same thing in different words. C2 says *the answer is then given, with an explanation of why it is chosen*, which describes the requirement rather than issuing it. Compliance is lowest under C2. The syntactic rewording changes voice, mood and clause order at once, so this observation cannot be narrowed to any one of them; Section 6.6 returns to that limit.

C3 is the one rewording of *C* that holds. It rewrites the prompt as a labelled list, with **EXPLAIN**: on its own line, and 84.9% of its answers still explain. It also produces the shortest answers of the four, at 77 output tokens against 117 for the canonical form, so the compliance is not bought with length.

6.5.3 The Direction Is Not Fixed

The most useful thing in Table 6.2 is that the signs differ. Every rewording of *B* improves adherence and every rewording of *C* damages it. Rewording is therefore not a source of degradation that a system should be protected from. It is a variable that can be set well or badly.

It follows that the published form of a prompt is not the best form of it. Baseline *B* is taken from a paper, and three separate rewritings of it, none of which changes what it asks for, all score higher than the original. A practitioner who copies a prompt from a paper is copying one point in a space, not the best point in it.

6.6 What These Results Do Not Show

The measured effects are large, but their reach is narrow, and four limits are worth stating directly.

The effect was found on adherence, which is the measure that had room to move. Faithfulness and relevance sat near their ceilings in every condition, so this experiment cannot say whether rewording affects them. A failure to detect a change on a measure with a range of five points is not evidence that no change is there. Section 5.8 gives the size of effect this design could have found.

The effect rests on two of the three baselines. Adherence is undefined for baseline *A*, and this is worth stating plainly rather than leaving it as a note on a table. *A* asks for one thing: *use the following pieces of context to answer the question*. That is grounding, and grounding is what faithfulness measures. An adherence rule for *A* would therefore be a second and cruder copy of a measure the design already has, and scoring it would count one failure twice and enter the two counts into the analysis as though they were independent. It would also flatter *A*: faithfulness runs near 0.95, so a grounding rule would pass almost always and *A* would appear to obey its prompt better than *B* and *C* on a rule it was never really tested against. The measure is therefore skipped rather than scored as satisfied, so that a condition never earns a free point for a requirement its prompt does not state.

The consequence is that the finding of Section 6.3 rests on baselines *B* and *C*. Baseline *A* is the shortest and plainest of the three prompts, and it is the one that cannot be tested on the measure where the whole effect appeared. Whether a prompt that asks for a single thing is as sensitive to rewording as one that asks for four is a question this design cannot answer, and it is not safe to assume the answer either way. The three rewordings of *A* were still estimated on the other three measures, and none of them moved one that survives correction. That says less than it sounds, because those measures sat at their ceilings or on the floor. One piece of evidence points the other way. On the secondary F1 measure of Table 6.3, the lexical rewording A1 lowers correctness in both cells, by 0.019 at the gold level and 0.022 at the retrieved level, and both intervals exclude zero. It is a small effect and it comes from a measure that is reported rather than decided on, so it settles nothing. It is enough to say that baseline *A* should not be assumed immune.

The adherence rules are narrow proxies for what the prompts ask. Baseline *C* asks the model to explain its choice, and the rule looks for a causal connective such as *because* or *therefore*. A connective is not an explanation. The count of markers reported above shows the rewordings genuinely stop the model from writing causal language, and not merely from writing one particular word, but whether the reasoning behind it is any good is not something this measure sees. The same holds for *B*, where a bracketed number is not a check that the cited passage supports the sentence. Faithfulness measures that separately, and it did not move.

Everything here is one generator, one judge, one corpus and one retrieval depth. The judge scored the two judged measures without a human agreement study behind it, as Section 5.7.2 notes, so those two measures carry no reliability estimate. The two measures the verdict rests on are program-checked and do not depend on the judge, which limits the damage, but the faithfulness result does depend on it.

The secondary correctness measure is not a measure of truth. Token F1 rewards overlapping words, so a fluent wrong answer that reuses the vocabulary of the question can score above a short correct one. It is a better proxy than containment on this data, which is a claim about containment, not a defence of F1.

7 Conclusion

7.1 Summary of Findings

This thesis asked how far systematic changes in the system prompt influence the behaviour and outcomes of a RAG system when the user input, the knowledge base, the retrieval mechanism and the underlying language model are held fixed. The answer is that they influence it a great deal, and that the size of the influence depends on which property of the answer is measured.

Twelve system prompts were built from three published baselines. Each baseline was written in four wordings: its canonical published form and three rewordings that change the vocabulary, the sentence structure or the layout. Every wording states the same requirements, so the only thing that varies across the four is φ . Each prompt was run over 100 MS MARCO queries at two evidence levels with three repetitions, which produced 6,768 answers, and every answer was scored on four measures.

The clearest result is on instruction adherence. Eleven of the twelve estimates exclude zero after correction, ten of them in both evidence levels, and the effects are large. Rewriting baseline *C* in the syntactic form costs it 46 points of adherence at the gold level, and rewriting it with synonyms costs 44. The requirements did not change in either case. Only the phrasing did.

The direction of the effect is not fixed. Every rewording of baseline *B* scored higher than its published form, and every rewording of baseline *C* scored lower. Rewording is therefore not a hazard to be guarded against but a parameter that can be set well or badly. It also follows that a prompt published with a system is not the best form of that prompt: three rewritings of baseline *B*, none of which changes what it asks for, all outperform the original.

On faithfulness and answer relevance no effect was found that survives correction, with one exception in one cell that does not replicate in the other. That is a weaker statement than it appears. Both measures sat near the top of their range in every condition, faithfulness never below 0.91, so there was little room for a rewording to move them. This design could not have detected a small change there, and Section 5.8 gives the size it could have detected.

Correctness produced a result about measurement rather than about prompts. Measured by containment, the form fixed before the run, no estimate excludes zero. Measured by token F1 on the same answers with the same estimator, thirteen of eighteen do. Containment asks whether the reference answer appears inside the generated answer word for word, which floors on this data and leaves the measure with almost nothing to detect. Both are reported and the preregistered measure keeps its place in the decision. Where two measures of one property disagree this sharply, the disagreement describes the instruments, and a null from a measure with no room to move is not evidence of absence.

Taken together, H_0 is rejected. Prompts that state the same requirements in different words do not produce the same behaviour from a RAG system. The wording of a system prompt is not a cosmetic property of it.

7.2 Contribution

The thesis makes four contributions.

The first is formal. Chapter 4 extends the formulation of Lewis et al. (2020) so that the system prompt is an explicit conditioning variable of the generator, and then splits it into a requirement set and a wording, $S = \rho(\Sigma, \varphi)$. That split is what makes the question answerable. Without it, changing a prompt changes what it asks for and how it asks at the same time, and no difference in behaviour can be assigned to either. With it, the wording becomes an independent variable that can be varied on its own, the assignment of a condition becomes an intervention $\text{do}(S = S_j)$, and the quantity of interest becomes an average treatment effect rather than an observed association.

The second is a design that holds the rest of the system fixed by construction rather than by assertion. Retrieval runs to completion and is written to disk before any prompt exists, so no execution path connects a prompt condition to what was retrieved. The context assembled for a query is drawn from a generator seeded on the query, the evidence level and the repetition, and never on the condition, so all twelve conditions receive an identical user turn. That property was checked on the collected data rather than assumed. The causal claim rests on these two facts, and both are enforced in code and tested.

The third is empirical. Prompt sensitivity has been shown for user prompts outside retrieval-augmented settings. This thesis measures it for the system prompt inside one, with the retriever, the corpus, the index and the generator held constant, and reports effects on instruction adherence that are large, bidirectional and replicated across two evidence levels. It also gives a practical result for anyone building such a system: the published form of a prompt is one point in a space of equivalent wordings, not the best point in it.

The fourth is methodological, and it was not planned. The preregistered correctness measure and a secondary measure of the same property, computed on the same answers, return opposite verdicts. Reporting both, and keeping the preregistered one in the decision, shows how far a conclusion about prompt sensitivity can depend on the instrument rather than on the system. Any study of this kind should check whether its measures have room to move before it reads a null as an absence of effect.

7.3 Future Work

Several directions follow from the limits of what was done here.

The results rest on one generator. Whether a rewording that helps one model helps another, or whether the direction of the effect is a property of the model rather than of the wording, is open. Repeating the design across model families and sizes would settle it, and the design supports it: the generator is a single fixed parameter and nothing else would change.

The judged measures carry no reliability estimate. Section 5.7.2 specifies an agreement study against human labels, and the sampling for it is implemented, but the labelling was not carried out. Completing it would put a bound on how far the judge itself is sensitive to the surface form of the answers it grades, which matters here more than usual, since the thesis assumes models are sensitive to their instructions and the judge is a model.

The adherence measures are narrow. A bracketed number is not a check that the cited passage supports the sentence, and a causal connective is not an explanation. Both rules were chosen because a rule can decide them, and both are proxies for a requirement that is broader than the rule. Measures that check whether a citation is warranted, or whether a justification is sound, would test the same prompts against what they actually ask for.

The three rewording axes each change several things at once. The lexical rewording of baseline *C* replaces many words, and the syntactic one changes voice, mood and clause order together. The data show that compliance falls furthest under the syntactic rewording, which is the one that turns a direct instruction into a description of what happens. That is a plausible reading, but the design cannot establish it, because voice was not varied on its own. Varying one linguistic feature at a time would turn the observation in Section 6.5 into a claim.

If published prompts are not optimal, the natural next question is how to find better ones. Searching over wordings with the requirement set held fixed is a well-defined problem, and the estimator used here already provides its objective.

Finally, the query set is short factoid web search. Section 2.4 motivates retrieval with settings such as clinical decision support and legal question answering, whose documents differ from web passages in length, structure and register. Whether an instruction that governs behaviour on MS MARCO governs it there is untested, and the effects reported here bound what may be expected in those settings without establishing it.

Bibliography

- Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. (2024). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*.
- Bush, V. (1945). As we may think. *The Atlantic Monthly*, 176(1):101–108.
- Cheerla, C. (2025). Advancing retrieval-augmented generation for structured enterprise and internal data.
- Chen, J., Lin, H., Han, X., and Sun, L. (2024). Benchmarking large language models in retrieval-augmented generation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38:17754–17762.
- Cheng, Z. and Mastropaolo, A. (2026). An empirical study on the effects of system prompts in instruction-tuned models for code generation.
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387.
- Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., and Larson, J. (2024). From local to global: A graph RAG approach to query-focused summarization.
- Es, S., James, J., Espinosa-Anke, L., and Schockaert, S. (2023). Ragas: Automated evaluation of retrieval augmented generation.
- Friel, R., Belyi, M., and Sanyal, A. (2024). RAGBench: Explainable benchmark for retrieval-augmented generation systems.
- Gao, L., Ma, X., Lin, J., and Callan, J. (2023a). Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, pages 1762–1777.
- Gao, T., Yen, H., Yu, J., and Chen, D. (2023b). Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., and Wang, H. (2023c). Retrieval-augmented generation for large language models: A survey.
- Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M. (2023). Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*.
- He, J., Zhang, B., Rouhizadeh, H., Chen, Y., Yang, R., Lu, J., Chen, X., Liu, N., and Teodoro, D. (2025). Retrieval-augmented generation in biomedicine: A survey of technologies, datasets, and clinical applications.

- Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70.
- Holtzman, A., Buys, J., Du, L., Forbes, M., and Choi, Y. (2020). The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*.
- Hua, A., Tang, K., Gu, C., Gu, J., Wong, E., and Qin, Y. (2025). Flaw or artifact? rethinking prompt sensitivity in evaluating LLMs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.
- Jeong, S., Baek, J., Cho, S., Hwang, S. J., and Park, J. C. (2024). Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), Volume 1: Long Papers*, pages 7036–7050.
- Jin, B., Zeng, H., Yue, Z., Yoon, J., Arik, S., Wang, D., Zamani, H., and Han, J. (2025). Search-R1: Training LLMs to reason and leverage search engines with reinforcement learning.
- Johnson, J., Douze, M., and Jégou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547.
- Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., and Yih, W.-t. (2020). Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781. Association for Computational Linguistics.
- Kostić, B., Fallon, C., Risch, J., and Löser, A. (2026). Same meaning, different scores: Lexical and syntactic sensitivity in LLM evaluation. To appear in the Proceedings of the 2026 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC).
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474.
- Lin, Y.-H., Chen, Q.-H., Cheng, Y.-J., Zhang, J.-R., Liu, Y.-H., Hsia, L.-Y., and Chen, Y.-N. (2025). LLM inference enhanced by external knowledge: A survey.
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.
- Lu, S., Schuff, H., and Gurevych, I. (2024). How are prompts different in terms of sensitivity? In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 5833–5856.
- Malkov, Y. A. and Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836.
- Mao, Y., Dong, X., Xu, W., Gao, Y., Wei, B., and Zhang, Y. (2024). FIT-RAG: Black-box RAG with factual information and token reduction.

- Mu, N., Lu, J., Lavery, M., and Wagner, D. A. (2025). A closer look at system prompt robustness.
- Nguyen, T., Rosenberg, M., Song, X., Gao, J., Tiwary, S., Majumder, R., and Deng, L. (2016). MS MARCO: A human generated MACHine Reading COMprehension dataset.
- Niu, C., Wu, Y., Zhu, J., Xu, S., Shum, K., Zhong, R., Song, J., and Zhang, T. (2024). RAGTruth: A hallucination corpus for developing trustworthy retrieval-augmented language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*.
- Pan, J. J., Wang, J., and Li, G. (2024). Survey of vector database management systems. *The VLDB Journal*, 33(5):1591–1615.
- Pandya, H. A. and Bhatt, B. S. (2021). Question answering survey: Directions, challenges, datasets, evaluation matrices.
- Pearl, J. (2009). *Causality: Models, Reasoning, and Inference*. Cambridge University Press, Cambridge, 2 edition.
- Pipitone, N. and Alami, G. H. (2024). LegalBench-RAG: A benchmark for retrieval-augmented generation in the legal domain.
- Qin, R., Wang, Q., Wang, T., Wei, Z., and Shen, W. (2026). Evaluating and explaining prompt sensitivity of LLMs using interactions.
- Robertson, S. and Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.
- Ruiz, F. J. and Cardona-Betancourt, V. (2026). Prompt carefully! ChatGPT displays rule-based insensitivity to contingencies. *Journal of Contextual Behavioral Science*, 41:101012.
- Saad-Falcon, J., Khattab, O., Potts, C., and Zaharia, M. (2024). ARES: An automated evaluation framework for retrieval-augmented generation systems. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 338–354.
- Salinas, A. and Morstatter, F. (2024). The butterfly effect of altering prompts: How small changes and jailbreaks affect large language model performance. In *Findings of the Association for Computational Linguistics: ACL 2024*.
- Salton, G., Wong, A., and Yang, C. S. (1975). A vector space model for automatic indexing. *Communications of the ACM*, 18(11):613–620.
- Sclar, M., Choi, Y., Tsvetkov, Y., and Suhr, A. (2024). Quantifying language models’ sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. In *International Conference on Learning Representations (ICLR)*.
- Singh, A., Ehtesham, A., Kumar, S., Khoei, T. T., and Vasilakos, A. V. (2025). Agentic retrieval-augmented generation: A survey on agentic RAG.
- Wahle, J. P., Ruas, T., Xu, Y., and Gipp, B. (2024). Paraphrase types elicit prompt engineering capabilities. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 11004–11033, Miami, Florida, USA. Association for Computational Linguistics.

- Wang, M., Yao, Y., Xu, Z., Qiao, S., Deng, S., Wang, P., Chen, X., Gu, J.-C., Jiang, Y., Xie, P., Huang, F., Chen, H., and Zhang, N. (2024). Knowledge mechanisms in large language models: A survey and perspective.
- Yan, S.-Q., Gu, J.-C., Zhu, Y., and Ling, Z.-H. (2024). Corrective retrieval augmented generation.
- Ying, J., Cao, Y., Xiong, K., Cui, L., He, Y., and Liu, Y. (2024). Intuitive or dependent? investigating LLMs’ behavior style to conflicting prompts. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*, pages 4221–4246.
- Yu, H., Gan, A., Zhang, K., Tong, S., Liu, Q., and Liu, Z. (2024). Evaluation of retrieval-augmented generation: A survey.
- Zhan, P., Xu, Z., Tan, Q., Song, J., and Xie, R. (2024). Unveiling the lexical sensitivity of LLMs: Combinatorial optimization for prompt enhancement. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5128–5154, Miami, Florida, USA. Association for Computational Linguistics.
- Zhang, B., Xin, H., Li, J., Zhang, D., Fang, M., Liu, Z., Nie, L., and Liu, Z. (2025). Benchmarking poisoning attacks against retrieval-augmented generation.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.
- Zheng, M., Pei, J., Logeswaran, L., Lee, M., and Jurgens, D. (2024). When “a helpful assistant” is not really helpful: Personas in system prompts do not improve performances of large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 15126–15154.
- Zhou, J., Lu, T., Mishra, S., Brahma, S., Basu, S., Luan, Y., Zhou, D., and Hou, L. (2023). Instruction-following evaluation for large language models.
- Zhuo, J., Zhang, S., Fang, X., Duan, H., Lin, D., and Chen, K. (2024). ProSA: Assessing and understanding the prompt sensitivity of LLMs. In *Findings of the Association for Computational Linguistics: EMNLP 2024*.